

GPU & AI Infrastructure Interview Handbook

从 GPU 硬件、CUDA、NCCL/RDMA 到 Kubernetes、vLLM 与平台系
统设计

Senior / Staff AI Infra • GPU Platform • Kubernetes Infra • LLM Inference

GPU 全景知识体系与面试手册

硬件 → 软件栈 → 并行计算 → 通信 → Kubernetes 管理 → LLM 推理 → 监控与排障

通用岗位: AI Infra / GPU Platform / MLOps / SRE / Backend Infra / Cloud Infra

本书适合解决的问题

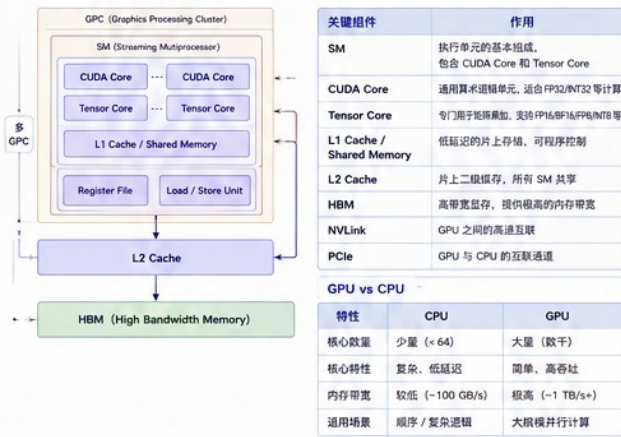
- ✓ 面试中关于 GPU 的高频问题
- ✓ 快速构建系统化 GPU 知识体系
- ✓ 排查 GPU 性能瓶颈与故障
- ✓ 设计高可用、高性能 GPU 平台

图例

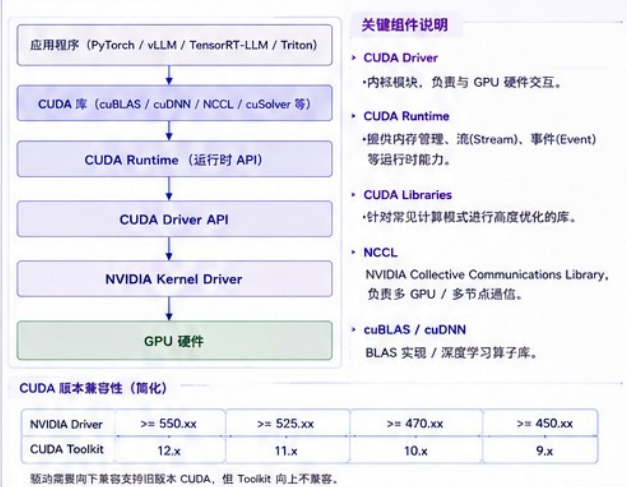
- 蓝色: 通用岗位
- 黄色: 芯片/框架
- 绿色: 软件层
- 红色: 网络通信层
- 深绿色: K8s 管理层
- 浅蓝色: 监控观测层

版本: v1.0 (2026-05)

1. GPU 硬件架构概览



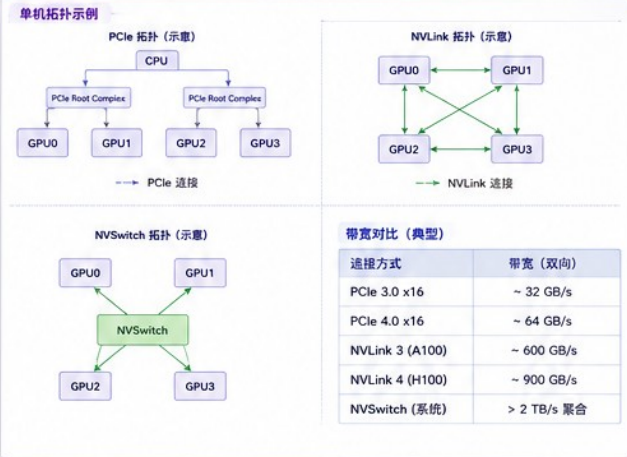
2. CUDA 软件栈



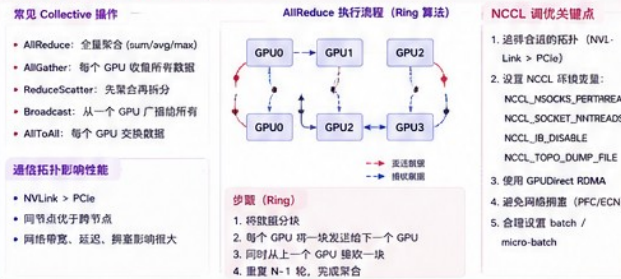
3. GPU 内存 (HBM) 详解



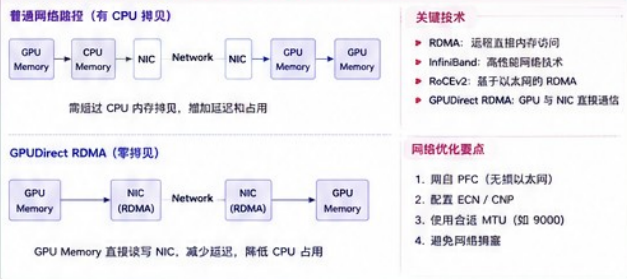
4. GPU 互联: PCIe / NVLink / NVSwitch



5. NCCL 与多 GPU 通信



6. 多节点通信: RDMA / InfiniBand / RoCE



7. Kubernetes 中的 GPU 管理



8. MIG / Time Slicing / DRA



使用方式

建议按照“概念 → 机制 → 图 → 指标/命令 → 故障场景 → 面试追问 → System Design”的顺序复习。不要只背名词，要能够从请求延迟一路追到 GPU/HBM/NCCL/RDMA/Kubernetes placement。

0. 知识地图与核心公式

层级	核心主题	你必须能回答的问题
硬件	SM / Warp / Tensor Core / HBM / L2	为什么 GPU 对矩阵计算有优势？瓶颈是 compute 还是 memory？
软件	Driver / Runtime / Libraries / Streams	容器里的 CUDA 为什么依赖 Host Driver？
互联	PCIe / NVLink / NVSwitch / NUMA	为什么 8 卡机器不同 GPU 组合性能不同？
通信	NCCL / RDMA / IB / RoCE	collective hang/slow 怎么定位？
Kubernetes	Device Plugin / Operator / MIG / DRA	Pod 怎样获得 GPU？如何做到共享、拓扑和健康管理？
Inference	Prefill / Decode / KV Cache / Batching	TTFT/TPOT/吞吐如何权衡？
Platform	Scheduler / Quota / Autoscaling / Observability	如何设计大规模 GPU 平台？

Weights Memory $\approx$ Parameters $\times$ bytes_per_parameter
KV Cache $\approx 2 \times$ Layers $\times$ KV_Heads $\times$ Head_Dim $\times$ Tokens $\times$ Batch $\times$ bytes
Service Capacity $\neq$ TFL0PS; always benchmark with real model, context, batch, precision and parallelism.

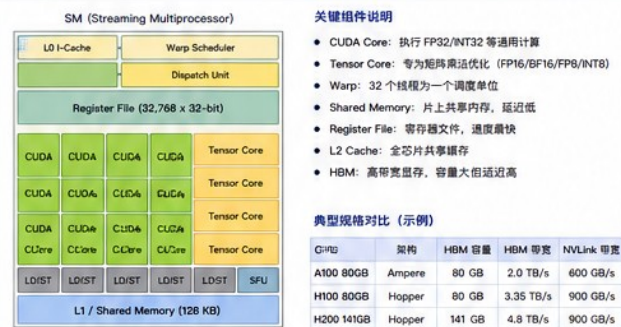
GPU 技术全景与 AI Infra 面试手册

从硬件原理到 Kubernetes 运维，再到 LLM 推理和系统设计

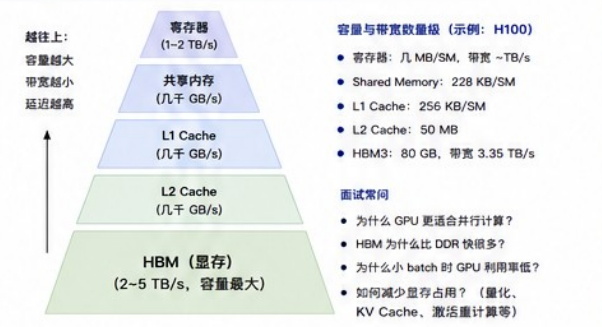
硬件架构 CUDA 栈 多 GPU 通信 K8s 集成 大模型推理 监控与排障 系统设计



1. GPU 硬件架构总览



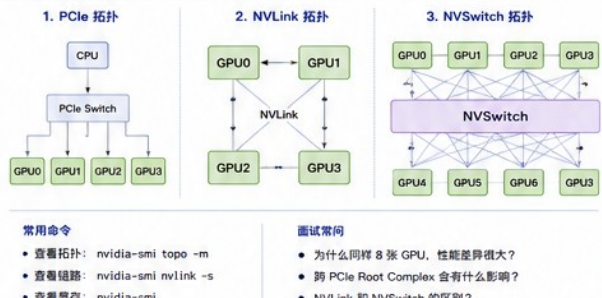
2. GPU 内存层次结构



3. CUDA 软件栈与执行流程



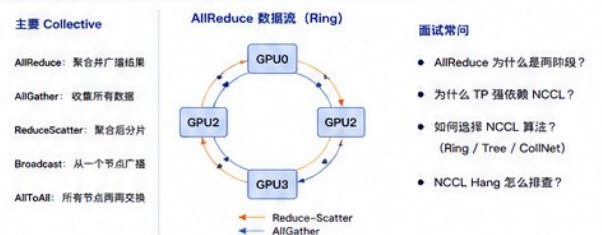
4. 多 GPU 拓扑结构



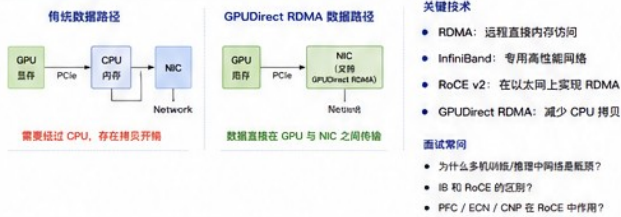
5. GPU 内存占用组成 (以大模型推理为例)



6. NCCL 通信原语与数据流



7. 网络与 GPUDirect RDMA



8. Kubernetes 中的 GPU 管理



9. MIG 与 Time Slicing



10. 大模型推理中的 GPU 计算流程



11. GPU 监控指标体系 (关键指标)



12. GPU 排障思路总览



图 0-1 GPU/AI Infra 面试全栈：硬件、CUDA、内存、网络、K8s、推理与排障

1. GPU 硬件架构：从晶体管到 SM / Tensor Core

本章目标是建立 GPU 的硬件 mental model。面试中无需背每一代芯片的精确参数，但必须能解释一条 kernel 如何被 warp 调度到 SM 上、数据如何从 HBM 进入计算单元，以及为什么 occupancy、memory bandwidth 与 tensor core 使用率会影响吞吐。

GPU 与 CPU 的设计目标

- CPU 强调低延迟、复杂控制流和强单线程性能；GPU 强调吞吐、并行度和高内存带宽。
- GPU 的核心抽象是 SIMT：大量线程以 warp 为基本调度单位执行同一 kernel。
- AI Infra 工程师不需要手写复杂 CUDA kernel，但必须理解 occupancy、warp、SM、HBM、Tensor Core 与性能瓶颈之间的关系。

面试表达建议

回答“GPU 与 CPU 的设计目标”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

SM、Warp 与执行资源

- SM (Streaming Multiprocessor) 是 GPU 的主要执行单元；内部包含 warp scheduler、寄存器文件、shared memory/L1、CUDA cores、Tensor Cores、load/store 单元等。
- warp 通常由 32 个线程组成。分支发散会降低一个 warp 的有效并行效率。
- Occupancy 受寄存器、shared memory、block size 等限制；高 occupancy 不等于高性能，但过低通常意味着无法隐藏延迟。

面试表达建议

回答“SM、Warp 与执行资源”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

Tensor Core 与精度

- Tensor Core 面向矩阵乘加。常见数据类型包括 FP32、TF32、FP16、BF16、FP8、INT8 等。
- 训练与推理会根据精度、数值稳定性、显存和吞吐在 BF16/FP16/FP8/INT8 间权衡。
- 面试重点不是背 FLOPS，而是解释：为什么降低精度能减少 HBM 占用、提高 tensor core 吞吐，并可能降低通信数据量。

面试表达建议

回答“Tensor Core 与精度”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

概念	作用	常见瓶颈信号
SM	执行 thread blocks/warps	occupancy 低、stall 高
Warp	32-thread 调度单位	branch divergence
Tensor Core	矩阵乘加	精度/shape 不匹配导致利用率低
HBM	主显存	bandwidth bound / OOM
L2	共享缓存	cache miss / memory traffic

2. GPU Memory: HBM、Cache、KV Cache 与 OOM

GPU OOM、低吞吐和长尾延迟往往最终都与显存有关。AI Infra 工程师应能估算权重、KV Cache 和 workspace，理解 HBM 容量与带宽是两个不同约束。

内存层次

- 寄存器最快但容量最小；shared memory/L1 为 SM 内低延迟存储；L2 为全 GPU 共享缓存；HBM 容量最大、延迟更高但带宽极大。
- 许多 LLM decode 工作负载是 memory-bandwidth bound，因此“GPU utilization 高”不一定表示算力吃满。

面试表达建议

回答“内存层次”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

显存构成

- 推理显存通常由模型权重、KV Cache、activation、CUDA context、通信/框架 workspace、临时 buffer 和碎片共同组成。
- 模型权重粗略估算：参数量 $\times$ 每参数字节数。70B BF16 仅权重约 140GB，通常需要多 GPU 或量化。
- KV Cache 随 batch、sequence length、layer 数、KV heads、head dimension 和 dtype 线性增长，是在线推理容量规划的关键。

面试表达建议

回答“显存构成”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

OOM 与碎片

- OOM 可能来自真实容量不足，也可能来自 allocator fragmentation、峰值 workspace、KV block 预留或多进程上下文。
- 排障时区分 allocated、reserved、free、framework cache、driver-level usage；不要只看 nvidia-smi 的单一数字。

面试表达建议

回答“OOM 与碎片”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

```
# 例：70B BF16 权重
70e9 parameters  $\times$  2 bytes  $\approx$  140 GB
# 还未包含 KV Cache、workspace、runtime、fragmentation
```


3. CUDA 软件栈与运行时

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

软件栈

- 典型路径：PyTorch/vLLM → CUDA libraries/runtime → Driver API → NVIDIA kernel driver → GPU。
- CUDA Toolkit 在容器内提供 runtime、nvcc 和用户态库；内核驱动通常安装在宿主机。
- Driver 与 CUDA Toolkit 存在兼容关系：面试要能解释为什么“容器 CUDA 版本”和“Host driver 版本”不是同一个概念。

面试表达建议

回答“软件栈”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

Streams、Events 与异步执行

- CUDA stream 提供按序命令流；不同 stream 可以在硬件允许时并发。
- H2D/D2H copy、kernel、通信可通过 stream 和 event 进行流水线重叠。
- 性能优化经常来自 overlap：计算与通信重叠、prefill 与 decode 调度、数据拷贝与 kernel 执行重叠。

面试表达建议

回答“Streams、Events 与异步执行”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

4. GPU 互联：PCIe、NVLink、NVSwitch 与 NUMA

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

拓扑决定性能

- PCIe 是 CPU/GPU 与设备间通用互联；NVLink/NVSwitch 提供更高 GPU-GPU 带宽。
- 同样 8 张 GPU，因为 PCIe root complex、NUMA、NVLink 拓扑不同，TP/NCCL 性能可能差异巨大。
- 常用命令：nvidia-smi topo -m。要结合 NIC、NUMA、GPU affinity 分析。

面试表达建议

回答“拓扑决定性能”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

Topology-aware placement

- Tensor Parallel workload 应优先放在互联更紧密的 GPU 集合上。
- 多节点时还要考虑 NIC affinity：GPU 到 RNIC 的 PCIe/NUMA 路径会影响 GPUDirect RDMA。

面试表达建议

回答“Topology-aware placement”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

```
nvidia-smi topo -m
nvidia-smi nvlink -s
numactl --hardware
lspci -tv
```

5. NCCL 与多 GPU Collective

NCCL 是多 GPU 系统的“数据平面”。很多所谓 GPU 性能问题最终是通信问题，因此要从 collective、拓扑、链路和网络四层理解。

Collective 基础

- NCCL 提供 AllReduce、AllGather、ReduceScatter、Broadcast、AllToAll 等 collective。
- 训练常用 AllReduce 同步梯度；TP 常用 AllReduce/AllGather；MoE Expert Parallelism 常出现 AllToAll。
- Ring、Tree、NVLS/CollNet 等算法/路径选择由拓扑、消息大小和运行环境共同影响。

面试表达建议

回答“Collective 基础”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

性能与故障

- NCCL Hang 常见原因包括 rank 不一致、网络丢包、RDMA/PFC/ECN 配置、MTU、GPU/NIC Xid、拓扑异常、进程 crash 或 collective 调用顺序不一致。
- 排障要同时看 NCCL_DEBUG、GPU Xid、IB/RoCE counters、交换机 telemetry 和应用 rank 状态。

面试表达建议

回答“性能与故障”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

```
NCCL_DEBUG=INFO
NCCL_DEBUG_SUBSYS=INIT,GRAPH,NET,COLL
# 配合 ibstat / ibv_devinfo / ethtool / dmesg / nvidia-smi
```

6. RDMA / InfiniBand / RoCE / GPUDirect RDMA

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

数据路径

- 传统路径可能经过 GPU→CPU memory→kernel/TCP→NIC；GPUDirect RDMA 允许 NIC 直接 DMA GPU memory，减少 CPU copy 与内核路径。
- InfiniBand 是专用 RDMA fabric；RoCEv2 在以太网上承载 RDMA，依赖更严格的拥塞与无损设计。

面试表达建议

回答“数据路径”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

RoCE 关键点

- PFC 可缓解特定优先级的丢包，但配置错误会引入 pause storm；ECN/CNP 用于拥塞反馈。
- 排障关注 MTU、PFC/ECN、buffer、ECMP、NIC firmware/driver、RDMA counters、PCIe topology。

面试表达建议

回答“RoCE 关键点”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

7. Kubernetes 中的 GPU 管理

Kubernetes 原生调度器并不了解 CUDA kernel，它管理的是资源声明和设备可用性。关键在于 Device Plugin/DRA 如何把硬件能力转换成 scheduler 可理解的资源，再由 runtime 将设备注入容器。

传统 Device Plugin 路径

- 节点安装 NVIDIA driver 和 container toolkit；Device Plugin 向 kubelet 注册并暴露 nvidia.com/gpu 等 Extended Resource。
- scheduler 基于整数资源做 placement；kubelet/runtime 最终把设备节点、环境变量、CDI 等注入 container。
- 传统 Extended Resource 简单可靠，但对复杂设备属性、共享、拓扑和动态配置表达能力有限。

面试表达建议

回答“传统 Device Plugin 路径”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

GPU Operator

- GPU Operator 用 operator 模式管理 driver、container toolkit、device plugin、DCGM exporter、MIG manager、NFD/GFD 等组件。
- 生产环境要考虑驱动升级、kernel 兼容、节点 drain、canary、GPU health quarantine、版本矩阵与回滚。

面试表达建议

回答“GPU Operator”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

```
resources:
  limits:
    nvidia.com/gpu: 1
# scheduler 只看到 Extended Resource；设备注入由 kubelet + device plugin/runtime 完成
```

8. MIG、Time Slicing、MPS 与 DRA

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

MIG

- MIG 在支持的 GPU 上进行硬件级分区，提供独立的显存和计算资源隔离，适合多租户和小模型。
- 代价是实例规格固定、拓扑和重配置复杂，且不是所有 workload 都适合切分。

面试表达建议

回答“MIG”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

Time Slicing 与 MPS

- Time Slicing 提供时间共享，隔离弱但使用简单；MPS 可改善多进程并发执行效率，但安全与隔离语义不同。
- 选择时要围绕隔离、利用率、SLO、租户模型和 workload shape。

面试表达建议

回答“Time Slicing 与 MPS”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

DRA

- Kubernetes DRA 自 v1.35 stable，使用 DeviceClass、ResourceSlice、ResourceClaim 等 API 让 scheduler/kubelet/driver 协同分配设备。
- DRA 更适合描述设备属性、偏好、共享与动态配置，是 GPU scheduling 未来的重要方向。

面试表达建议

回答“DRA”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

方案	隔离	灵活性	典型场景
整卡	高	低	大模型/高 SLO
MIG	较高	中	多租户/小模型
Time Slicing	低	高	低优先级共享
DRA	取决于 Driver	高	复杂设备声明/拓扑/共享

9. LLM Inference: Prefill、Decode、Batching 与 KV Cache

LLM Serving 的关键不是“把模型跑起来”，而是如何在 HBM、compute、batch、queue、KV Cache 和延迟 SLO 之间动态平衡。

Prefill vs Decode

- Prefill 对 prompt 做大规模并行计算，通常 compute-heavy；Decode 每步生成 token，常受 HBM bandwidth、KV Cache 和小 batch 影响。
- TTFT 主要受排队、prefill、冷启动等影响；TPOT/ITL 主要受 decode 调度、batch、KV cache、通信等影响。

面试表达建议

回答“Prefill vs Decode”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

Continuous Batching

- 持续批处理会动态把请求加入/移出 batch，提高 GPU 利用率；核心 trade-off 是 throughput 与 tail latency。
- Chunked Prefill 可把长 prompt 分块，与 decode 更细粒度混合，缓解长 prefill 对在线请求的阻塞。

面试表达建议

回答“Continuous Batching”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

vLLM

- vLLM 的关键能力包括 Paged/Block KV Cache、scheduler、continuous batching、chunked prefill、prefix caching 等。
- 容量规划要同时看 max_num_seqs、batched tokens、KV cache headroom、model parallel size、GPU memory utilization 和真实 traffic distribution。

面试表达建议

回答“vLLM”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

指标	含义	常见优化方向
TTFT	首 token 延迟	queue、prefill、cold start、router
TPOT/ITL	输出 token 间隔	decode batch、HBM、NCCL
tokens/s	吞吐	batch、并行、量化、kernel
KV usage	KV cache 压力	并发、context、block management

10. 分布式推理并行策略：TP / PP / DP / EP

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

Tensor Parallelism

- 把矩阵/层内计算拆到多 GPU；优点是单模型可跨卡，缺点是每层频繁通信，对 NVLink/NCCL 敏感。

Pipeline Parallelism

- 按层/Stage 划分模型；减少单层 collective，但会引入 pipeline bubble 和 stage imbalance。

Data Parallelism

- 多 replica 处理不同请求，扩展吞吐；在线 serving 常与请求 router、负载均衡和 KV locality 结合。

Expert Parallelism

- MoE 将 expert 分散到 GPU/节点，通常大量依赖 AllToAll；网络设计和 expert load balance 关键。

11. GPU Observability: 从 DCGM 到服务级 SLO

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

硬件层

- SM utilization、Tensor Core utilization、HBM used/bandwidth、power、temperature、clocks、PCIe/NVLink throughput、ECC、Xid。

面试表达建议

回答“硬件层”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

通信层

- NCCL collective latency、RDMA throughput、retransmit/timeout、PFC pause、ECN/CNP、NIC errors。

面试表达建议

回答“通信层”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

服务层

- QPS、tokens/s、TTFT、TPOT/ITL、queue depth、batch size、KV cache usage/hit ratio、preemption、cold-start。

面试表达建议

回答“服务层”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

原则

- 不要把 GPU utilization 当作唯一 KPI。必须用 end-to-end SLO + GPU + communication + scheduler queue 共同解释性能。

面试表达建议

回答“原则”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

```
nvidia-smi dmon
dcgm-exporter -> Prometheus/Grafana
# 关联服务指标: TTFT, TPOT, tokens/s, queue depth, KV cache usage
```


12. GPU 故障排查 Runbook

GPU 故障排查最忌讳直接重启。Staff 级回答要体现分层定位：应用 → 推理引擎 → GPU → 通信 → 主机 → Kubernetes → 网络。

GPU OOM

- 先确认模型权重、KV Cache、workspace、batch/sequence、fragmentation；再看是否存在多进程上下文或 replica 竞争。
- 策略：降低 batch/seq、量化、调整 KV cache、增加 TP、减少并发、使用更大 HBM GPU。

面试表达建议

回答“GPU OOM”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

GPU 不可见

- 检查 nvidia-smi、driver module、container toolkit、device plugin、CDI/设备节点、Pod spec、runtime class、kubelet allocatable。

面试表达建议

回答“GPU 不可见”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

NCCL Hang

- 确认所有 rank 存活与 collective 顺序；检查 NCCL_DEBUG；验证 intra-node NVLink 与 inter-node RDMA；排查 MTU/PFC/ECN/Xid。

面试表达建议

回答“NCCL Hang”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

性能下降

- 从应用 SLO → batch/queue → GPU kernel/HBM → NCCL → RDMA → CPU/NUMA/PCIe → Kubernetes placement 分层定位。

面试表达建议

回答“性能下降”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

症状	第一层检查	第二层	常见根因
OOM	HBM usage/KV	allocator/requests	context 太长、并发过高、碎片
低吞吐	batch/queue	HBM/SM/NCCL	decode memory bound、通信瓶颈
NCCL Hang	rank/process	RDMA/Xid	网络/collective 顺序/硬件
Pod Pending	scheduler event	allocatable/plugin	GPU 不足、label/taint、设备健康

13. GPU Capacity Planning 与成本模型

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

容量公式

- 先估权重 + KV Cache + activation/workspace，再留 10%-25% operational headroom。
- 吞吐容量不能只用理论 FLOPS；必须使用目标模型、上下文长度、batch、量化和并行策略做 benchmark。

面试表达建议

回答“容量公式”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

扩容指标

- GPU utilization 适合作为辅助，不适合作为唯一 autoscaling 指标；更推荐 queue depth、waiting tokens、TTFT、KV pressure 与 admission rate。
- 节点层扩容需要考虑 GPU 节点冷启动、driver/daemon readiness、镜像与模型加载时间。

面试表达建议

回答“扩容指标”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

14. GPU Scheduler / Platform System Design

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

关键问题

- 资源模型：整卡/MIG/共享/DRA；调度：GPU type、HBM、NVLink、NUMA、NIC affinity；队列：priority、quota、fairness、gang；生命周期：健康、隔离、升级、回收。
- 模型服务：router、replica、autoscaler、model cache、warm pool、load shedding、admission control。
- 平台安全：租户隔离、device access、host privilege、secrets、image provenance、sandbox。

面试表达建议

回答“关键问题”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

Staff 级 Trade-off

- 利用率 vs 延迟；共享 vs 隔离；TP scale-up vs DP scale-out；单集群 vs 多集群；统一 GPU 池 vs 按 SKU/拓扑分池。

面试表达建议

回答“Staff 级 Trade-off”时，先用 20-30 秒定义概念，再讲数据路径或控制路径，最后给出一个生产场景和 trade-off。

```
Request -> Router -> Admission -> Model Pool -> GPU Pod -> GPU/NCCL/RDMA
          |
          +--> Autoscaler -> K8s Scheduler -> GPU Node Provisioner
Observability spans every layer; capacity and SLO must be closed-loop.
```

15. 高频面试题与追问树

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

核心题

- 为什么 GPU 适合 LLM?
- SM、warp、Tensor Core、HBM 分别是什么?
- 为什么模型能加载但并发一上来就 OOM?
- CUDA Driver 和 Toolkit 的区别?
- PCIe、NVLink、NVSwitch 的区别?
- NCCL AllReduce 为什么会成为瓶颈?
- InfiniBand 与 RoCEv2 有什么区别?
- GPUDirect RDMA 如何减少 CPU 开销?
- K8s Pod 是如何得到 GPU 的?
- Device Plugin 与 DRA 有什么区别?
- MIG 和 Time Slicing 如何选择?
- TP/PP/DP/EP 怎么选?
- TTFT 和 TPOT 分别受什么影响?
- GPU utilization 95% 但 tokens/s 低，怎么排查?
- 如何设计 1000 GPU 的推理平台?

16. 30 天学习与实验路线

本章从基础概念、内部机制、工程权衡和面试追问四个角度展开，重点是建立可用于生产排障和系统设计的 mental model。

Week 1: 硬件与 CUDA

- GPU architecture、memory hierarchy、warp/SM、Tensor Core、CUDA driver/runtime、nvidia-smi、Nsight 基础。

Week 2: 通信与 K8s

- PCIe/NVLink、NCCL、RDMA/RoCE、Device Plugin、GPU Operator、MIG、DRA。

Week 3: Inference

- vLLM、KV Cache、Continuous Batching、TP/PP/DP、benchmark、TTFT/TPOT。

Week 4: 平台与面试

- DCGM observability、capacity、autoscaling、故障演练、System Design、mock interview。

附录 A：GPU 面试速查大图

GPU 全栈知识体系与面试指南

从硬件到推理，从单机到集群，面试必备的完整知识地图与实践手册



硬件架构



软件栈



多GPU通信



Kubernetes



推理框架



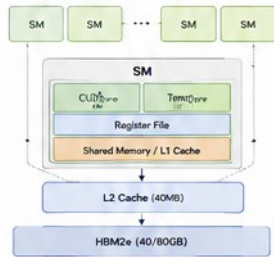
监控运维

1. GPU 硬件架构总览

GPU 是大规模并行计算的处理器，专为高吞吐、低延迟的矩阵运算设计。

- SM (Streaming Multiprocessor)**
执行单元，每个SM包含CUDA Core、Tensor Core、寄存器、共享内存等
- CUDA Core**
通用计算单元，适合FP32/INT2等
- Tensor Core**
专为矩阵乘法优化，支持FP16/BF16/FP8等精度
- HBM (High Bandwidth Memory)**
高带宽显存，提供远高于GDDR的带宽
- L2 Cache**
大容量缓存，减少访问HBM的延迟
- NVLink / NVSwitch**
GPU间的高速互联，带宽远高于PCIe
- PCIe**
GPU与CPU的连接通道

典型GPU架构 (以NVIDIA A100为例)



GPU vs CPU 对比

指标	CPU	GPU
核心数	少 (8~128)	多 (数千)
优化目标	低延迟，强单线程	高吞吐，大规模并行
内存带宽	较低 (GB/s)	极高 (TB/s)
适合场景	复杂逻辑、串行任务	矩阵计算、并行计算

2. CUDA 软件栈

CUDA 是 NVIDIA 提供的并行计算平台和编程模型。

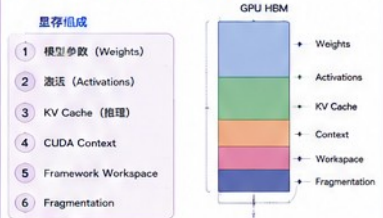


关键概念

- Driver**: 在 Host 上运行，负责与 GPU 硬件交互
- Toolkit**: 提供编译器、库、工具 (nvcc, profiler 等)
- Runtime**: 应用程序运行时使用的动态库 (libcudart)
- 容器化**: 通常包含 Runtime/Library, Driver 由 Host 提供

3. GPU 内存模型

显存 (HBM) 是有限资源，需合理规划和估算。



显存估算公式

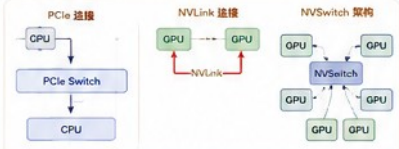
模型参数显存 = 参数量 × 数据类型字节数
KV Cache 大小 = $2 \times L \times H \times D \times S \times \text{数据类型字节数}$
L: 层数 H: KV Head 数 D: Head 维度 S: 序列长度 (tokens)

示例 (70B 模型, BF16, KV Cache)

- 模型权重: $70B \times 2 \text{ bytes} = 140 \text{ GB}$
- 假设 L=80, H=64, D=128, S=2048, BF16 (2 bytes)
- KV Cache = $2 \times 80 \times 64 \times 128 \times 2048 \times 2 \text{ bytes} = 2.68 \text{ GB} / \text{序列}$
- 如果 batch size = 32, 总 KV Cache = 85.8 GB

4. GPU 互联技术

单机内 GPU 互联

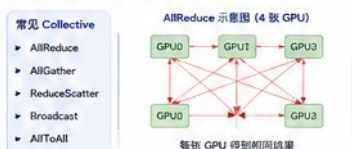


带宽对比 (典型值)

连接方式	带宽 (双向)	延迟	特点
PCIe 4.0 x16	32 GB/s	~1us	通用, 成本低
NVLink 3	50 GB/s	~0.2us	GPU-GPU 高带宽
NVLink 4	112.5 GB/s	~0.2us	A100/H100 支持
NVSwitch (8-way)	900 GB/s+	~0.2us	多 GPU 全互联

5. 多 GPU 通信与 NCCL

NCCL 提供高性能的 GPU 集合通信原语。



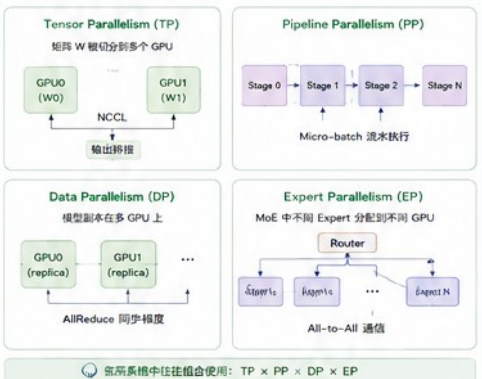
通信层次



性能影响因素

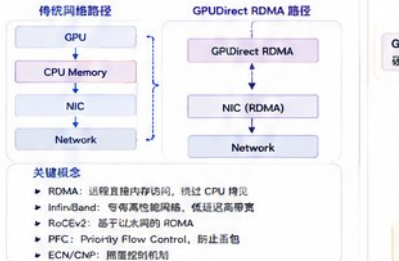
- 拓扑结构 (NVLink > PCIe)
- NCCL 算法选择 (Ring / Tree / ColNet 等)
- 网络带宽与延迟 (IB/RoCE)
- 消息大小 (小消息延迟敏感, 大消息带宽敏感)
- 开发通信与计算重叠 (Overlap)

6. 分布式训练/推理并行策略



业界系统中往往组合使用: TP × PP × DP × EP

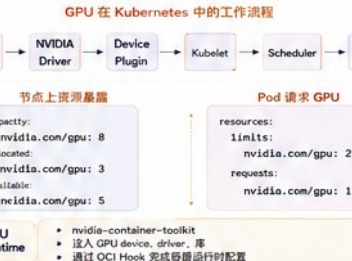
7. 网络与 RDMA 技术



关键概念

- RDMA**: 远程直接内存访问, 绕过 CPU 拷贝
- InfiniBand**: 专为高性能网络, 低延迟高带宽
- RoCEv2**: 基于以太网的 RDMA
- PFC**: Priority Flow Control, 防止丢包
- ECN/CNP**: 拥塞控制机制

8. Kubernetes 中的 GPU 管理



节点上资源暴露

Capacity: nvidia.com/gpu: 8
Allocated: nvidia.com/gpu: 3
Available: nvidia.com/gpu: 5

Pod 请求 GPU

resources: limits: nvidia.com/gpu: 2
requests: nvidia.com/gpu: 1

GPU Runtime

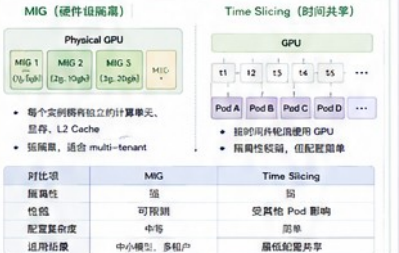
- nvidia-container-toolkit
- 注入 GPU device, driver, 库
- 通过 OCI Hook 完成容器运行时配置

9. NVIDIA GPU Operator



优势: 统一管理 GPU 节点软件栈, 简化运维, 提高稳定性

10. MIG 与 Time Slicing



对比项

隔离性	MIG	Time Slicing
性能	强	弱
配置复杂度	可预期	受其他 Pod 影响
适用场景	简单	最低配置要求

11. DRA: 更强大的设备资源管理 (未来方向)



优势

- 支持多策略选择 (显存、拓扑、型号等)
- 支持全生命周期管理 (Allocate/Prepare/Cleanup)
- 支持可插拔资源, 扩展灵活

示例: 选择特定显存和拓扑的 GPU

```
apiVersion: resource.k8s.io/v1beta1
kind: ResourceClaim
spec:
  resourceClassName: gpu-class
  parameters:
    attributes:
      nvidia.com/memory: "≥20Gi"
      topology.kubernetes.io/zone: us-east-1a
```

12. 推理服务中的 GPU 使用 (以 vLLM 为例)



vLLM 核心优化

- Continuous Batching (持续批处理)
- Paged KV Cache (页式 KV Cache)
- Prefix Cache (前缀缓存)
- Chunked Prefill (分块 Prefill)
- 异步调度 (Async Scheduling)

关键指标

- TFTT (Time To First Token)
- TPOT (Time Per Output Token)
- 吞吐量 (tokens/sec)
- 并发数
- KV Cache 命中率
- GPU 利用率

16. 面试高频问题示例



常用命令

```
nvidia-smi
nvidia-smi dmon
nvidia-smi topo -m
nvidia-smi query -d
dcmgr metrics
```

关键监控指标

- SM 利用率
- HBM 利用率 / 带宽
- Tensor Core 利用率
- 温度 / 功耗
- PCIe / NVLink 带宽
- ECC 错误 / Xid 错误

14. 常见 GPU 问题与排查思路



问题分类

- OOM (显存耗尽)
- 性能低 (吞吐低)
- NCCL 通信问题
- 驱动 / Xid 错误
- 网络问题 (RDMA)
- 调度不合理

15. GPU 容量规划与估算



单请求显存估算公式

$Mem_{total} = Mem_{weights} + Mem_{KV} + Mem_{activations} + Mem_{workspace} + Mem_{context} + Mem_{fragmentation}$

并发容量估算

并发数 = (显存总量 - 系统开销) / 单请求显存占用

影响因素

- 模型大小
- 数据类型 (FP16/BF16/FP8/INT8)
- 序列长度 (S)
- 批大小 (batch)
- 并发数
- TP/PP/OP 配置
- 硬件估计

附录 B：常用命令清单

类别	命令	用途
GPU	nvidia-smi / nvidia-smi dmon	健康、显存、利用率、功耗、温度
Topology	nvidia-smi topo -m	GPU/CPU/NIC/NVLink 拓扑
DCGM	dcgmi diag / dcgm-exporter	诊断与 Prometheus 指标
PCIe	lspci -tv / lspci -vv	PCIe 设备与链路
NUMA	numactl --hardware	NUMA topology
RDMA	ibstat / ibv_devinfo / rdma link	IB/RDMA 状态
NIC	ethtool -S / ip -s link	链路 counters
K8s	kubectl describe node/pod	GPU allocatable、调度事件
NCCL	NCCL_DEBUG=INFO	collective/init/network 调试

附录 C：Research / 官方资料

- NVIDIA CUDA Programming Guide — <https://docs.nvidia.com/cuda/cuda-programming-guide/>
- NVIDIA CUDA Best Practices Guide — <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- NVIDIA GPU Operator — <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/>
- NVIDIA GPU Operator MIG / DRA documentation — <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/>
- NCCL User Guide & Diagnostics — <https://docs.nvidia.com/deeplearning/nccl/user-guide/>
- Kubernetes Dynamic Resource Allocation — <https://kubernetes.io/docs/concepts/resource-management/dynamic-resource-allocation/>
- vLLM Scheduler Documentation — <https://docs.vllm.ai/en/latest/api/vllm/config/scheduler/>

版本说明

GPU 硬件规格、CUDA/NCCL、GPU Operator、Kubernetes DRA 和 vLLM 都在快速演进。面试准备时应优先掌握稳定的原理与数据路径，再针对目标公司的硬件/版本做最后一轮校准。

附录 D：60 道 GPU / AI Infra 高频面试题（含回答框架）

D.1 为什么 GPU 比 CPU 更适合 LLM？

推荐回答：从吞吐优化、SIMT、大量并行 ALU、Tensor Core、HBM 带宽解释；补充 CPU 仍负责控制面、tokenization、调度和部分算子。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.2 什么是 SM？

推荐回答：SM 是 GPU 的主要执行单元，负责 warp 调度并拥有寄存器、shared memory/L1、CUDA/Tensor Core 等资源。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.3 warp 为什么是 32 个线程？

推荐回答：这是 NVIDIA SIMT 的硬件调度粒度；面试重点是 warp divergence 和 memory coalescing 的影响。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.4 Tensor Core 和 CUDA Core 的区别？

推荐回答：CUDA Core 是通用标量/向量算术单元；Tensor Core 专门加速矩阵乘加，支持多种低精度格式。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.5 什么是 occupancy？

推荐回答：活跃 warp 相对硬件上限的比例；受寄存器、shared memory、block size 约束，高 occupancy 不等于高性能。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.6 HBM capacity 和 bandwidth 有什么区别？

推荐回答：capacity 决定能否装下权重/KV；bandwidth 决定 memory-bound workload 每秒能搬多少数据。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.7 为什么 decode 常是 memory bound?

推荐回答：每生成一个 token 要反复读取大量权重/KV，而单步计算规模较小，算术强度低。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.8 70B BF16 模型显存怎么估?

推荐回答：仅权重约 140GB，再加 KV、workspace、runtime、fragmentation；通常需要多卡或量化。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.9 KV Cache 为什么会并发 OOM?

推荐回答：其大小随并发、context、layer、KV heads、head dim 线性增长；并发增加时可迅速吃满 HBM。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.10 什么是 GQA/MQA?

推荐回答：减少 KV heads，从而显著降低 KV Cache 和带宽压力，是现代 LLM 推理常见优化。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.11 CUDA Driver 和 Runtime 区别?

推荐回答：Driver API 更底层；Runtime API 更高层。Host kernel driver 与容器内 userspace toolkit 也要区分。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.12 为什么容器里装 CUDA 仍需要宿主机 Driver?

推荐回答：用户态 CUDA 通过宿主机 NVIDIA kernel driver 操作真实 GPU。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.13 CUDA stream 是什么?

推荐回答：异步命令队列；同 stream 保序，不同 stream 可并发并用于 copy/compute/communication overlap。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。

- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.14 Pinned memory 有什么作用？

推荐回答：页锁定主机内存便于 DMA，提高 H2D/D2H copy 效率，但使用过多会压缩 OS 可分页内存。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.15 PCIe 和 NVLink 的区别？

推荐回答：PCIe 通用且跨设备；NVLink 面向高带宽 GPU-GPU 互联，适合 TP/NCCL。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.16 NVSwitch 解决什么？

推荐回答：为多 GPU 提供交换式 NVLink fabric，改善任意 GPU 间高带宽互联。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.17 为什么 NUMA 对 GPU 有影响？

推荐回答：CPU、GPU、NIC 挂载在不同 NUMA/PCIe root 下，跨 NUMA 访问会增加延迟、降低带宽。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.18 nvidia-smi topo -m 怎么看？

推荐回答：关注 GPU-GPU 连接类型、CPU affinity、NIC proximity，判断 TP/NCCL/RDMA 放置质量。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.19 NCCL 是什么？

推荐回答：面向 NVIDIA GPU 的 collective communication library，提供 AllReduce/AllGather/ReduceScatter/AllToAll 等。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.20 AllReduce 为什么常拆成 ReduceScatter + AllGather？

推荐回答：可分阶段聚合并分发，便于 ring 等算法充分利用链路带宽。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.21 Ring vs Tree 怎么选？

推荐回答：Ring 带宽利用高适合大消息；Tree 延迟更低适合小消息/更多节点；NCCL 会根据拓扑自动选择。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.22 TP 为什么通信敏感？

推荐回答：层内拆分导致几乎每层都需要 collective，因此对 NVLink/NCCL 延迟和带宽非常敏感。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.23 EP 为什么依赖 AllToAll？

推荐回答：MoE token 需要路由到不同 expert，导致大规模 all-to-all 数据交换。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.24 GPUDirect RDMA 的核心收益？

推荐回答：NIC 直接 DMA GPU memory，减少 CPU copy、内核路径和延迟。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.25 InfiniBand 和 RoCE 的区别？

推荐回答：IB 是专用 fabric；RoCE 在以太网上承载 RDMA，通常依赖 PFC/ECN 等拥塞机制。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.26 PFC 有什么风险？

推荐回答：配置不当可能造成 pause propagation / head-of-line blocking / pause storm。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.27 ECN/CNP 的作用？

推荐回答：检测拥塞并通知发送端降速，避免持续队列堆积。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.28 NCCL Hang 如何排查？

推荐回答：rank 存活→collective 顺序→NCCL 日志→GPU Xid→NVLink→RDMA→交换机 counters。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.29 K8s Device Plugin 如何工作？

推荐回答：向 kubelet 注册并上报设备；kubelet 通过 Allocate 获取设备注入信息，scheduler 只看到 Extended Resource。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.30 为什么 GPU request 通常等于 limit？

推荐回答：Extended Resource 没有 CPU 那种可压缩 request/limit 语义，通常以整数设备分配。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.31 GPU Operator 做什么？

推荐回答：自动化 driver、toolkit、device plugin、DCGM、MIG manager、NFD/GFD 等组件生命周期。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.32 MIG 和 Time Slicing 怎么选？

推荐回答：需要硬件隔离/SLO 选 MIG；追求共享率且容忍弱隔离可用 time slicing。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.33 MPS 和 Time Slicing 有什么不同？

推荐回答：MPS 允许多进程更高效并发共享执行资源；time slicing 更接近时间片轮转。

- 继续追问时补充一个生产故障或性能案例。

- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.34 DRA 相比 Device Plugin 的优势？

推荐回答：更丰富的设备属性/偏好/共享/动态配置表达，使用 DeviceClass/ResourceSlice/ResourceClaim。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.35 为什么 topology-aware scheduling 很重要？

推荐回答：GPU/NIC/NVLink/NUMA 的物理位置直接影响 TP 和 RDMA 性能。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.36 什么是 gang scheduling？

推荐回答：一组 Pod 需要同时满足最小资源后再启动，避免分布式 job 部分占卡后死锁或浪费。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.37 GPU quota 怎么做？

推荐回答：按 team/project 定额并配 priority/fairness/queue；关键是避免碎片与长期占用。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.38 Prefill 和 Decode 的区别？

推荐回答：Prefill 高并行、偏 compute；Decode 逐 token、偏 memory/latency。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.39 Continuous Batching 为什么提升吞吐？

推荐回答：动态填充 batch，减少 GPU 空泡并提高并行度。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.40 Chunked Prefill 解决什么？

推荐回答：长 prompt 分块，减少一次 prefill 对 decode 的阻塞，改善 tail latency。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.41 TTFT 和 TPOT 如何分别优化？

推荐回答：TTFT：排队/冷启动/prefill/router；TPOT：decode batch/HBM/KV/NCCL。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.42 Prefix Cache 有什么价值？

推荐回答：相同前缀可复用 KV，降低重复 prefill 计算，但需要命中率和内存管理。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.43 为什么 GPU utilization 高但 tokens/s 低？

推荐回答：可能 memory bound、NCCL bound、poor batching、小 kernel、power/thermal throttle 或队列策略问题。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.44 如何判断 compute bound 还是 memory bound？

推荐回答：结合 kernel profiler、SM/Tensor utilization、HBM bandwidth、算术强度与 roofline 思路。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.45 为什么温度/功耗会影响性能？

推荐回答：达到 thermal/power limit 后会降频，表现为 utilization 高但实际吞吐下降。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.46 Xid error 是什么？

推荐回答：NVIDIA driver 报告的 GPU 异常类别；需结合具体 Xid、dmesg、DCGM 判断硬件/驱动/应用。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.47 ECC 错误怎么处理？

推荐回答：区分 corrected/uncorrected，关注增长趋势与 retired pages；严重时隔离节点并走硬件维护。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.48 GPU 节点为什么要做 quarantine？

推荐回答：防止有问题的 GPU 继续被 scheduler 分配，避免反复影响 workload。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.49 GPU autoscaling 为什么难？

推荐回答：节点冷启动慢、驱动/daemon/model load 重，且 GPU 指标噪声大；需使用队列/SLO 预测与 warm pool。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.50 GPU 节点池如何分层？

推荐回答：按 SKU、NVLink 拓扑、网络能力、租户/SLO、spot/on-demand 分池。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.51 TP scale-up 和 DP scale-out 如何取舍？

推荐回答：TP 解决单模型显存但通信强；DP 提升吞吐且通信少，但每 replica 需要完整模型。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.52 如何估算 inference capacity？

推荐回答：真实 benchmark 得出 tokens/s 或 req/s，结合 traffic 分布、SLO、headroom 和失败域做容量。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.53 如何设计多模型 GPU 平台？

推荐回答：model router + admission + per-model pool + cache/warmup + quota + autoscaling + observability。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.54 如何做 GPU 驱动升级？

推荐回答：canary 节点→drain→driver/operator 升级→health/benchmark→逐批 rollout→回滚计划。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.55 为什么模型加载很慢？

推荐回答：镜像/模型下载、磁盘吞吐、解压、CPU、网络、权重初始化和 GPU upload 都可能是瓶颈。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.56 GPUDirect Storage 是什么？

推荐回答：让存储到 GPU 的数据路径减少 CPU bounce buffer，适合高吞吐数据加载场景。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.57 GPU 平台最关键的 SLO？

推荐回答：不仅是 availability，还应覆盖 TTFT/TPOT、queue time、tokens/s、job start latency、GPU health。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.58 如何做端到端 tracing？

推荐回答：请求 ID 贯穿 gateway/router/vLLM，关联 Pod/GPU/rank/NCCL/RDMA 指标和日志。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

D.59 Staff 级 GPU System Design 先讲什么？

推荐回答：先定义 workload/SLO/规模，再讲资源模型、placement、network、serving、autoscaling、failure domain、observability 和成本。

- 继续追问时补充一个生产故障或性能案例。
- 如果涉及性能，至少给出一个可观测指标和一个验证命令。
- 最后说明 trade-off，而不是把某个方案描述为绝对最优。

附录 E：15 个 Hands-on Labs

E.1 GPU 拓扑实验

目标：运行 `nvidia-smi topo -m`、`lspci -tv`、`numactl --hardware`，画出 GPU/CPU/NIC 拓扑。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.2 CUDA 基础

目标：运行简单 `vector add / matmul`，观察不同 `block size` 与 `stream`。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.3 HBM/显存实验

目标：用不同 `batch/context` 运行模型，记录权重、KV、`reserved/allocated` 的变化。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.4 NVLink 实验

目标：用 `NCCL tests` 对比 NVLink 与 PCIe 路径带宽。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.5 NCCL 多卡

目标：运行 `all_reduce_perf`，改变 `message size`，画出 `bandwidth/latency` 曲线。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.6 RDMA 基础

目标：检查 `ibstat/rdma link/ibv_devinfo`，并验证 GPU-NIC affinity。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.7 K8s Device Plugin

目标：部署 GPU pod，追踪 `Node allocatable`→`scheduler`→`kubelet`→`container device`。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.8 GPU Operator

目标：查看 Operator 创建的 DaemonSet/CRD/labels，理解组件依赖。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.9 MIG

目标：在实验节点配置 MIG，观察资源名称与 Pod 调度。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.10 Time Slicing

目标：配置共享 GPU，压测多个 Pod 的延迟与公平性。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.11 DRA

目标：搭建支持 DRA 的测试集群，创建 DeviceClass/ResourceClaim 并观察调度生命周期。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.12 vLLM Benchmark

目标：改变 max_num_seqs/context/batch，记录 TTFT、TPOT、tokens/s 与 KV usage。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.13 TP Benchmark

目标：比较 TP=1/2/4 的吞吐与 NCCL traffic，解释 scale efficiency。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.14 GPU OOM Runbook

目标：人为制造长 context/高并发 OOM，建立可重复诊断步骤。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

E.15 完整 System Design

目标：设计一个多租户 1000-GPU inference platform，输出架构图、SLO、容量公式、故障域和成本策略。

- 记录实验假设、命令、指标、结果与结论。
- 至少保存一张拓扑/监控截图或 benchmark 表格。
- 写出如果结果与预期不一致，你会从哪三层继续排查。

最终版扩展：GPU / AI Infrastructure 深度专章

本部分在基础手册之上进一步覆盖 CUDA 执行与性能分析、GPU 内存分配器、NCCL/RDMA 深入机制、Kubernetes GPU 调度与 DRA、vLLM 内部结构、容量工程、生产故障和 Staff-Level System Design。目标不是记住名词，而是能够从硬件、内核、运行时、网络、Kubernetes 和模型服务六个层面建立统一 mental model。

GPU & AI Infrastructure Interview Handbook (最终版)

v2.0 Final

从硬件到大规模推理，从 Kubernetes 调度到分布式通信与排障的完整知识体系

2026-05



用户请求



CUDA



Communication



Kubernetes



Inference



Inference



Observability

1. GPU 硬件体系结构

1.1 NVIDIA GPU 典型架构 (以 Hopper GH200 为例)

SM (Streaming Multiprocessor)

执行核心

- CUDA Core (标准计算)

- Tensor Core (矩阵计算)

- RT Core (光线追踪)

存储层次

- Register (速度最快, 容量最小)

- Shared Memory / L1 Cache

- L2 Cache (全芯片共享)

- HBM (超高带宽显存)

关键指标 (示例)

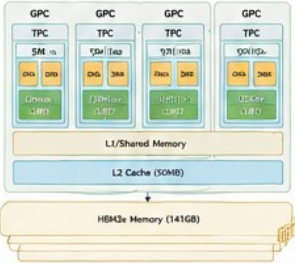
- FP16 Tensor TFLOPS: 989

- HBM3e 带宽: 4.8 TB/s

- L2 Cache: 50 MB

- NVLink 带宽: 900 GB/s

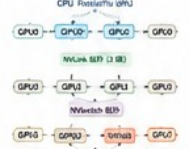
- 功耗 (TDP): 700W



1.2 GPU vs CPU 对比

	CPU	GPU
制程节点	少量超大核心	大量中等核心 (SMT)
应用	10-96	数千-数万
内存带宽/延迟	100 GB/s 级	> 3 TB/s 级 (HBM)
Latency	低	高
功耗/启动	低 (单核/轻量)	高 (并行/重载)
适用场景	通用串行任务	大规模并行/矩阵计算

1.3 GPU 拓扑示意



2. GPU Memory 深入理解

2.1 显存组成 (Inference 场景)



2.2 显存估算公式

模型权重
 $Mem_{weights} = P \times Bytes_per_param$

精度 Bytes/Param

FP32	4
BF16 / FP16	2
INT8	1
FP8	1
INT4	0.5

KV Cache (近似)
 $Mem_{kv} = 2 \times L \times H_{kv} \times D_{kv} \times B \times T \times Bytes$

L: 层数
H: KV 头数
D: Dims 每个头维度
B: Batch Size
T: 最大序列长度

示例 (70B, BF16, B=8, T=4096, L=80, H_{kv}=8, D_{kv}=128)

- 模型权重 = 70B × 2 bytes = 140 GB

- KV Cache = 2 × 80 × 8 × 128 × 8 × 4096 × 2B = 32.0 GB

- 其他开销 = 6 GB

- 总计 = 178 GB (单卡 80GB 不足, 需要并行/更低精度)

2.3 常见 OOM 原因

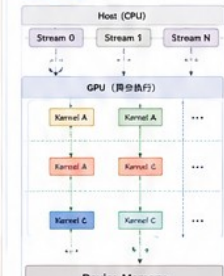
模型过大, 序列过长, KV Cache 膨胀, 并行化, 显存泄漏, 碎片化过大

3. CUDA 软件栈与执行模型

3.1 CUDA 软件栈



3.2 执行模型



3.2 BIOR CUDA 组件

- cuBLAS: 高性能线性代数
- NCCL: 多 GPU 通信库
- CUDA Stream: 异步执行队列
- CUDA Event: 事件同步
- Pinned Memory: 页锁定内存加速 H2D/D2H
- Unified Memory: 统一内存 (UVM)
- CUDA Graph: 减少内核启动开销

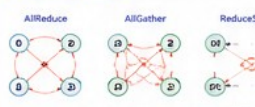
3.4 CUDA 版本兼容性

Driver 版本	支持的 CUDA Toolkit
≥ 550.x	12.5 / 12.6
≥ 535.x	12.2 / 12.3
≥ 525.x	12.0 / 12.1
≥ 515.x	11.x

Driver 向下兼容, 但 Toolkit 需 Driver 支持才能

4. 多 GPU 通信与 NCCL

4.1 Collective 通信模式



4.2 Ring vs Tree

Ring (环形拓扑): 适合大规模 GPU 数量, 但收敛慢。
Tree (树状拓扑): 收敛快, 但需要更多通信带宽。

4.3 常见问题

- 网络延迟 / PFC 阻塞
- GPU P2P 不可用
- Rank 不一致
- 网络配置错误
- 驱动/固件不兼容

4.4 NCCL Hang 排查流程

1. 确认 NCCL_DEBUG=INFO
2. 检查所有进程是否进入通信
3. 检查网络: 使用 nvidia-netstat
4. 检查网络 ID / PFC / ECN
5. 检查 GPU 拓扑与 P2P
6. 使用 nccl-tests 验证
7. 使用 nvidia-smi topo -m
8. 使用 NCCL 2.31 Diagnostics

5. 高性能网络: RDMA / InfiniBand / RoCE

5.1 数据路径对比



5.2 RoCEv2 关键技术

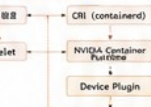
- PFC (Priority Flow Control): 流量控制 (避免拥塞)
- ECN (Explicit Congestion Notification): 显式拥塞通知
- GHP (Congestion Notification Packet): 拥塞通知包

5.3 网络指标

- IB Port Xmit Data / Rcv Data
- Rcv Pkts / Xmit Pkts
- Rcv Errors / Xmit Discards
- Link Downed
- PFC Pause
- ECN Marked
- RoCE Retransmit

6. Kubernetes 中 GPU 管理

6.1 GPU Pod 生命周期



6.2 关键组件

组件	作用
NVIDIA Driver	驱动, 内核模块
nvidia-container-toolkit	容器 GPU 资源管理
NVIDIA Device Plugin	向 kube 暴露 GPU 资源
DCGM - Exporter	GPU 指标采集
GPU Feature Discovery	GPU 特性发现
NFD (Node Feature Discovery)	节点特性发现

6.3 GPU Operator 架构



6.4 CDI (Container Device Interface)

CDI 规范定义了容器如何访问设备。通过 Device Plugin 和 CSI 接口实现。

7. GPU Sharing: MIG / Time Slicing / MPS

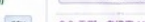
7.1 MIG (Multi-Instance GPU)



7.2 Time Slicing (时间共享)



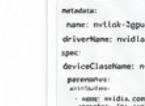
7.3 MPS (Multi-Process Service)



7.4 对比

	MIG	Time Slicing	MPS
隔离性	强	低	中
性能稳定性	高	低	中
易用性	中	高	高
适用场景	多租户隔离	轻量级共享	多进程共享

8.1 核心关系图



8.2 示例: 申请两个 NVLink 的 GPU

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaimTemplate
metadata:
  name: nvidia-gpu
  namespace: nvidia
spec:
  deviceClassName: nvidia.com/gpu
  parameters:
    apiVersion: nvidia.com/v1
    kind: ResourceClaim
    metadata:
      name: claim-1
    spec:
      resourceClaimTemplateName: nvidia-gpu
      status:
        allocated:
          devices:
            - name: "GPU-0"
            - name: "GPU-1"
            status: allocated
          count: 2
```

9. LLM Inference 与 GPU 利用

9.1 推理流程



9.2 KV Cache 管理



图：GPU & AI Infrastructure 最终版知识地图

最终版阅读方法

第一遍建立全栈路径；第二遍针对薄弱层做实验；第三遍使用面试树口述；最后用 System Design 和故障案例验证是否真正理解。

F. CUDA 执行模型与性能工程

F.1 从 CPU Launch 到 GPU Kernel 完成

一次 CUDA kernel 执行不是“CPU 调 GPU”这么简单。Host 线程调用 CUDA Runtime API, Runtime/Driver 创建或复用 context, 把 kernel launch 描述写入 stream, 驱动通过 command buffer 提交到 GPU。GPU 前端再把 thread block 分派到 SM, SM 内部以 warp 为调度单位执行。

Host Thread
-> CUDA Runtime API
-> CUDA Driver API
-> NVIDIA Kernel Driver
-> GPU Command Processor
-> GPC/TPC/SM
-> Thread Block
-> Warp (32 threads)
-> CUDA/Tensor Core
-> Memory hierarchy

概念	必须理解	常见性能问题
Grid / Block / Thread	Grid 由 blocks 组成, block 由 threads 组成; block 是 SM 调度/驻留的重要单位	block 太小无法填满 SM; 太大导致 register/shared memory 压力
Warp	NVIDIA 常见 warp=32 threads, 是 SIMT 执行基本单位	branch divergence 使不同路径串行化
Occupancy	active warps / theoretical max warps	高 occupancy 不等于高性能, 但过低通常无法隐藏 latency
Register	每线程私有、延迟极低	register spill 到 local memory 会显著变慢
Shared Memory	block 内共享的 on-chip memory	bank conflict、容量占用降低 occupancy
Streams	相互独立的 work queue/ordering domain	错误同步导致 copy/compute 无法 overlap

F.2 Warp、SIMT 与 Branch Divergence

SIMT 允许一个 warp 中的 32 个线程执行同一条指令, 但每个线程拥有自己的寄存器和数据。当一个 warp 内线程进入不同分支, 硬件通常需要对不同分支路径分别执行并屏蔽不相关线程, 造成有效执行槽浪费。

if (threadIdx.x % 2 == 0) { path_A(); } else { path_B(); }
同一个 warp 内一半走 A、一半走 B -> divergence
优化思路: 让分支边界尽量与 warp 边界对齐, 或使用数据重排/branchless 技巧

F.3 Memory Coalescing、Cache 与 Roofline

GPU 性能经常不是由 TFLOPS 决定, 而是由“算术强度 Arithmetic Intensity”决定。Roofline 模型把性能上限理解为 min(峰值计算能力, 内存带宽 × 算术强度)。Decode 阶段大量模型常呈 memory-bandwidth-bound, 而大 batch prefill 更可能逐渐接近 compute-bound。

瓶颈	症状	关键指标	优化方向
Compute-bound	Tensor Core/SM 长时间饱和	SM util、Tensor Core util、achieved FLOPS	更高效 kernel、FP8/BF16、fusion
Memory-bandwidth-bound	SM busy 但算力未滿, HBM 带宽高	DRAM throughput、L2 hit、bytes/FLOP	提高复用、量化、fusion、batching
Latency-bound	大量小 kernel / launch gap	kernel duration、launch latency、GPU idle gaps	CUDA Graph、kernel fusion、batch
Communication-bound	多卡 scaling 差	NCCL time、NVLink/RDMA BW	拓扑、collective、并行策略

F.4 CUDA Stream、Event、Graph

- Stream：同一 stream 中保持顺序，不同 stream 之间可以并发，但是否真的并发取决于硬件资源和依赖。
- Event：用于建立跨 stream 依赖和测量 GPU 时间。
- CUDA Graph：将大量重复 launch 预先捕获为图，减少 CPU launch overhead，适合重复稳定执行路径。

F.5 Nsight Systems 与 Nsight Compute

工具	回答什么问题	典型用途
Nsight Systems	系统时间线发生了什么？CPU/GPU/IO/NCCL 是否重叠？	发现 launch gap、同步、通信等待
Nsight Compute	单个 kernel 为什么慢？	occupancy、warp stall、memory throughput、Tensor Core
PyTorch Profiler	框架 op 与 CUDA kernel 如何对应？	模型级热点、CPU/GPU operator
nvidia-smi/dmon	节点是否健康和大致忙碌？	快速 triage，不适合作为精细 profiler

F.Q1 : GPU utilization 95% 就表示 GPU 用得好吗？

2 分钟回答

不一定。utilization 通常表示采样窗口内 GPU 是否有 kernel active，它不能告诉你 kernel 是否高效。还要结合 SM efficiency、Tensor Core、HBM bandwidth、kernel duration、occupancy 和应用层 tokens/s。

深入回答：如果是 decode-heavy LLM，GPU 可能持续有小 kernel 运行而显示高 utilization，但 HBM 带宽、batching 或 launch overhead 仍可能限制吞吐。判断“用得好”必须从业务吞吐和硬件效率同时看。

- 指标：SM Active、DRAM BW、Tensor Core、kernel gap、tokens/s、TPOT
- 验证命令：nvidia-smi dmon；nsys profile；ncu --set full
- Trade-off：更大 batch 能提高吞吐，但通常牺牲 TTFT/尾延迟和显存。
- Interviewer 追问：如何区分 compute-bound 和 memory-bound？；为什么 decode 更容易 memory-bound？；Nsight 里你会先看什么？

G. GPU Memory、Allocator 与 OOM 深入

G.1 存本：模型加到在推理

在线推理显存不能只按 weights 估算。最终需要考虑模型权重、KV Cache、临时 activation、CUDA context、communication buffer、framework workspace、graph capture、allocator reserved memory 和 fragmentation。

$$\begin{aligned} \text{Mem_total} \approx & \text{Mem_weights} + \text{Mem_KV} + \text{Mem_activation} + \text{Mem_workspace} \\ & + \text{Mem_context} + \text{Mem_comm} + \text{Mem_graph} + \text{Fragmentation} + \text{SafetyMargin} \end{aligned}$$

G.2 权重容量估算

格式	每参数近似字节	70B 权重理论值	工程注意点
FP32	4	~280 GB	通常不用于大模型 serving
BF16/FP16	2	~140 GB	最常见高精度 serving
FP8	1	~70 GB	需要硬件/模型/框架支持
INT8	1	~70 GB	量化策略影响精度与 kernel
INT4	0.5	~35 GB	解量化开销与 kernel 支持很关键

G.3 KV Cache 估算

标准 attention 的近似 KV cache 可按 $2(K+V) \times \text{layers} \times \text{KV heads} \times \text{head_dim} \times \text{tokens} \times \text{batch} \times \text{dtype_bytes}$ 计算。对于 GQA/MQA, KV heads 数可能远小于 query heads, 因此 KV 成本显著降低。

$$\begin{aligned} \text{KV_bytes} &\approx 2 \times L \times H_{kv} \times D_{head} \times T \times B \times \text{bytes_per_element} \\ \text{例: } L=80, H_{kv}=8, D=128, T=8192, B=32, \text{BF16 (2B)} \\ \text{KV} &\approx 2 \times 80 \times 8 \times 128 \times 8192 \times 32 \times 2 \approx 80 \text{ GiB 量级} \end{aligned}$$

G.4 Caching Allocator 与 Fragmentation

深度学习框架通常不会在每次 tensor 释放时立即 cudaFree, 而会使用 caching allocator 保留显存 block, 以降低频繁分配释放的同步和驱动成本。因此 reserved memory 可能显著大于 allocated memory。OOM 既可能是真正容量不足, 也可能是碎片化、峰值临时 buffer 或并发变化导致。

现象	可能原因	验证	处理
allocated 低但 reserved 高	缓存池保留	框架 memory summary	调整 allocator / 清理空闲 block (谨慎)
长短请求混合后 OOM	碎片化 + KV 波动	reserved-allocated、block distribution	paged KV、请求分桶、限制 max context
TP 增大后仍 OOM	KV/activation 不一定按 TP 理想缩放	per-rank memory	重新计算权重/缓存/通信 buffer
首次高并发突然 OOM	CUDA Graph/Workspace 峰值	启动期显存曲线	warmup 后设安全水位

G.5 Pinned Memory、Unified Memory 与 DMA

- Pinned/Page-locked host memory 使 DMA 传输更直接, 也更容易与计算 overlap; 但过量 pinned memory 会伤害主机内存管理。
- Unified Memory 提供统一虚拟地址和按需迁移, 对易用性友好, 但 page migration 可能带来不可预测抖动, 生产 LLM serving 通常更强调显式数据布局。

- GPUDirect RDMA 允许 NIC 直接访问 GPU memory，减少 CPU staging 和内存复制。

G.Q1：模型明明能加载，为什么并发一上来就 OOM？

2 分钟回答

因为“能加载”只证明权重和少量 runtime state 能放下。在线请求增加后，KV Cache、activation、临时 workspace、通信 buffer 和 allocator fragmentation 都会增长。

深入回答：我会先做显存账本：静态权重、每 token KV 成本、每请求 context、并发数和框架预留，再观察 allocated/reserved/KV usage 的实时变化，确认是容量问题还是碎片问题。

- 指标：HBM used、KV cache usage、allocated/reserved、preemption/eviction、OOM count
- 验证命令：nvidia-smi；framework memory_summary；vLLM metrics
- Trade-off：提高 KV 使用率可以提高吞吐，但安全余量减少，长尾请求更容易触发 preemption/OOM。
- Interviewer 追问：如何算 KV Cache？；GQA 为什么省 KV？；paged KV cache 解决什么问题？；reserved 与 allocated 有什么区别？

H. NCCL Collective、算法与多 GPU Scaling

H.1 Collective 语义

Collective	输入/输出	典型场景	通信特点
AllReduce	每 rank 输入，所有 rank 得到 reduction 结果	训练梯度同步、部分 TP	常由 ReduceScatter + AllGather 实现
AllGather	每 rank 一块，所有 rank 收集全部块	TP 激活拼接	输出随 world size 增大
ReduceScatter	聚合后每 rank 得到一片	TP/训练	与 AllGather 互补
AllToAll	每 rank 与所有 rank 交换不同片	MoE Expert Parallel	对网络 bisection bandwidth 极敏感
Broadcast	一个 root 发送给所有	参数/状态传播	根节点带宽与算法重要

H.2 Ring vs Tree

Ring 算法通常能很好利用链路带宽，适合大消息；Tree 能降低步骤深度，在小消息/低延迟场景更有优势。现代 NCCL 会根据拓扑、消息大小、协议和硬件自动选择 algorithm/protocol，实际还有 CollNet/NVLS 等路径。

```
Ring AllReduce (概念):
ReduceScatter: N-1 steps -> 每个 rank 得到结果分片
AllGather:      N-1 steps -> 交换分片得到完整结果

Tree: 深度近似 O(log N)，但每层链路利用和拓扑映射决定实际表现。
```

H.3 NCCL 初始化为什会 Hang

- Rank/world size 配置不一致，某个进程没有进入相同 collective。
- Bootstrap 网络不可达、hostname/interface 选择错误。
- GPU P2P/NVLink 拓扑或 IOMMU/ACS 配置导致 P2P 路径异常。
- RDMA device/interface/GID 配置错误。
- 防火墙、Security Group、NetworkPolicy 或 host firewall 阻断。
- 某个 GPU Xid/ECC/driver 状态异常，导致 rank 停滞。

H.4 2026 NCCL Diagnostics

NCCL 2.31 已提供内置 diagnostics。RAS diagnostics 可比较各 rank 的 GPU inventory、driver、ECC、NVLink 与 NCCL 环境一致性；Active diagnostics 可以在 communicator 初始化期间实际验证 P2P/通信路径。生产 runbook 应优先利用这些机制，而不是一开始盲目抓包。

```
NCCL_RUN_DIAGNOSTICS=1 <application>
# RAS (默认监听通常为 localhost:28028, 可配置)
echo diagnostics | nc localhost 28028
echo verbose status | nc localhost 28028
```

H.5 Scale Efficiency

```
ScaleEfficiency(N) = Throughput(N GPUs) / (N × Throughput(1 GPU))
例如单卡=1000 tok/s, 8 卡=6000 tok/s -> efficiency=6000/(8×1000)=75%
```

扩卡不一定线性：模型并行会引入通信和同步，某些工作负载还受 CPU、tokenization、网络、KV Cache 或 router 限制。系统设计时必须先说明扩卡的目的：容量放不下、单请求 latency、还是全局 throughput。

H.Q1 : 为什么 TP=8 可能比 TP=4 更慢？

2 分钟回答

TP 增大降低每张 GPU 的矩阵规模，但增加 collective 频率、通信量和同步点。如果跨 NVSwitch/跨 NUMA/甚至跨节点，通信成本可能超过计算节省。

深入回答：我会比较单请求 latency、NCCL 占比、message size、NVLink/RDMA 带宽、GPU topology 和 per-rank compute。TP 是“模型切分策略”，不是免费的扩容。

- 指标：NCCL duration、busbw/algbw、NVLink BW、RDMA BW、SM util、tokens/s
- 验证命令：nccl-tests all_reduce_perf; nvidia-smi topo -m; NCCL_DEBUG=INFO
- Trade-off: 更高 TP 可支持更大模型或降低单卡权重，但通信、故障域和调度难度上升。
- Interviewer 追问：Ring 和 Tree 怎么选？；跨节点 TP 最大风险是什么？；如何计算 scale efficiency?

I. RDMA / InfiniBand / RoCE 生产级理解

I.1 数据路径

传统路径: GPU -> PCIe -> Host Memory -> CPU/Kernel -> NIC -> Network
GPUDirect RDMA: GPU Memory <-> PCIe <-> RDMA NIC -> Fabric -> RDMA NIC <-> GPU Memory

RDMA 价值不仅是“快”，而是低 CPU 参与、低拷贝和低延迟。对多节点训练/推理，GPU-NIC topology 与 NIC/fabric 配置会直接影响 collective 性能。

I.2 InfiniBand vs RoCEv2

维度	InfiniBand	RoCEv2
网络	专用 IB fabric	以太网 + UDP/IP
拥塞/无损	IB 原生机制	通常依赖 PFC/ECN/DCQCN 等
路由	IB fabric manager	标准 L3 IP 路由能力
运维	HPC/AI 专用体系	可复用 DC Ethernet，但调优更复杂
常见问题	port/link/subnet manager	PFC storm、ECN、buffer、MTU、hashing

I.3 RoCE 的 PFC、ECN、CNP

- PFC：按 priority 暂停链路，避免 buffer overflow 丢包；错误配置会造成 pause storm / head-of-line blocking。
- ECN：交换机拥塞时标记包而非直接丢弃。
- CNP：接收端/网络反馈拥塞，发送端降低速率。
- 真正稳定的 RoCE 需要端到端一致的 QoS、MTU、queue mapping、ECN/PFC 和 buffer 规划。

I.4 GPU-NIC Affinity

如果 GPU 与 NIC 位于不同 NUMA socket/PCIe root complex，GPUDirect 路径可能跨 CPU interconnect，导致更高 latency 和更低 bandwidth。因此 Kubernetes topology-aware scheduling 不应只看“有几张 GPU”，还要考虑 GPU-GPU、GPU-NIC、NUMA 距离。

```
nvidia-smi topo -m
lspci -tv
numactl --hardware
ibdev2netdev
ibv_devinfo
rdma link
ethtool -S <iface>
```

I.5 RDMA Troubleshooting Tree

```
NCCL/RDMA 慢或 hang
├─ GPU 健康? Xid/ECC/clock/power
├─ GPU P2P/NVLink 正常? topo / p2p test
├─ NIC link/port/up? rdma link / ibstat
├─ GID/route/interface 对? NCCL_IB_* / ip route
├─ MTU 一致?
├─ PFC/ECN counters 异常?
├─ drops/retrans/ECN marked?
└─ nccl-tests 与 ib_write_bw 是否都慢? -> 定位 NCCL 层还是 fabric 层
```


J. Kubernetes GPU Scheduling : 从 Extended Resource 到 DRA

J.1 Device Plugin 完整链路

```
GPU Node
-> NVIDIA Driver
-> NVIDIA Device Plugin register(kubelet deviceplugin socket)
-> ListAndWatch(): advertise healthy devices
-> kubelet updates Node Capacity/Allocatable nvidia.com/gpu
-> scheduler Filter: extended resource fits?
-> kubelet DeviceManager Allocate()
-> env/devices/mounts/CDI edits
-> CRI -> containerd -> OCI runtime -> container sees GPU
```

Scheduler 传统上只知道 extended resource 数量，并不知道每张 GPU 的丰富属性和复杂拓扑。因此“2 张 GPU”可能被选成拓扑很差的一对。

J.2 DRA 对象与生命周期

```
DeviceClass -----+
                    |
ResourceClaimTemplate -> ResourceClaim
                    |
ResourceSlice <---- DRA Driver advertises devices
                    |
Pod references Claim -> Scheduler selects device/node
                    |
Kubelet PrepareResources -> DRA Driver
                    |
Container Start
                    |
Pod delete -> Unprepare/Cleanup
```

Kubernetes DRA 在 v1.35 已 stable。它允许通过 Kubernetes API 表达 device class、attributes、selectors、claims 等，比传统 opaque integer extended resource 更适合 GPU/NIC/accelerator 等复杂设备。

J.3 Topology-Aware Scheduling

需求	只看 GPU count 的风险	理想调度信息
TP=2	可能选跨 PCIe root 的两卡	NVLink/NVSwitch peer bandwidth
Multi-node TP	GPU 与 NIC 远	GPU-NIC NUMA/PCIe affinity
MoE EP	跨交换机 all-to-all 拥塞	fabric topology / network locality
MIG	只知道 profile, 不知道 parent GPU placement	MIG parent/NUMA/topology

J.4 Gang Scheduling / Queue / Fairness

分布式训练或多节点推理通常要求多个 Pod/worker 一起可用。如果只启动一部分，会占着 GPU 等另外一部分，造成资源死锁或低利用率。Gang scheduling 的核心是“要么整体满足，要么不占用”。在共享 GPU 集群还需要 queue、quota、priority、preemption 与 fairness。

- Kueue/Volcano 等负责 workload queueing、quota 和 admission。
- Kubernetes scheduler 负责最终 Pod -> Node placement。
- Cluster Autoscaler/Karpenter 负责节点容量，但 GPU node cold start 很慢。
- 三层控制器必须避免各自独立导致 oscillation。

J.5 GPU Node Pool 设计

维度	设计建议
SKU	按 H100/H200/B200/L4 等能力分 node pool，避免 scheduler 隐式混用
Topology	区分 NVLink/NVSwitch、NIC 数量、NUMA
Lifecycle	driver/operator canary，节点 drain，健康 quarantine
Autoscaling	warm pool + predictive + queue-aware
成本	on-demand/reserved/spot 分层；容错 workload 可用 spot
安全	多租户隔离、device exposure、privileged 组件最小化

J.Q1 : Pod 请求 2 张 GPU，scheduler 到底知道哪两张 GPU 吗？

2 分钟回答

传统 Device Plugin + extended resource 模式下，scheduler 主要做 nvidia.com/gpu 数量匹配，具体 device ID 通常在 kubelet DeviceManager Allocate 阶段确定。因此 scheduler 对设备属性与拓扑的表达能力有限。

深入回答：如果需要更丰富的 device selection，应关注 DRA、拓扑标签/自定义 scheduler/plugin，或 vendor/device-specific 方案。关键是分清 Node placement 和具体 device allocation 两个阶段。

- 验证命令：kubectl describe node；kubectl get resourceclaims/resourceslices；kubelet/device-plugin logs
- Trade-off：更智能的拓扑调度提高性能，但增加 scheduler/device control plane 的复杂度和碎片化风险。
- Interviewer 追问：Device Plugin Register/ListAndWatch/Allocate 分别做什么？；DRA 为什么比 extended resource 强？；如何做到 NVLink-aware scheduling？

K. GPU Sharing、虚拟化与多租户隔离

K.1 MIG / Time Slicing / MPS / vGPU 对比

机制	隔离	性能可预测性	典型用途	风险
MIG	硬件级 compute/memory partition	高	多租户小模型、稳定 QoS	profile 固定导致碎片
Time Slicing	时间共享，同一物理 GPU	低-中	低利用率任务、开发环境	显存/性能争用
CUDA MPS	进程服务复用 GPU context/执行	中	多进程计算共享	隔离不如 MIG
vGPU	虚拟化栈切分/调度 GPU	中-高	VM/KubeVirt/VDI/企业虚拟化	许可与栈复杂
Whole GPU	物理卡独占	最高	高性能训练/推理	利用率可能偏低

K.2 Multi-Tenant Threat Model

- 资源 DoS：一个租户耗尽 HBM、SM、PCIe/NVLink 或 host pinned memory。
- 数据隔离：模型权重、KV Cache、临时 buffer、日志/metrics 不应跨租户泄露。
- 设备节点：/dev/nvidia* 暴露必须通过 runtime/device mechanism 控制。
- 特权管理组件：Device Plugin、Driver daemon、DCGM 等具备高权限，需要单独安全边界。
- 调度侧信道：共享硬件会带来性能干扰和潜在 side-channel，需要根据安全等级选择共享模式。

L. vLLM Internals : Scheduler、Paged KV Cache 与 Serving

L.1 请求执行路径

```
HTTP Request
-> API Server / tokenizer
-> EngineCore
-> Scheduler queue
-> KVCacheManager admission/allocation
-> model runner
-> Prefill / Decode GPU kernels
-> sample token
-> scheduler next step
-> streaming response
```

L.2 Continuous Batching

静态 batching 要等一批请求一起开始/结束，会被长请求拖住。Continuous batching 在每个 iteration 重新决定当前有哪些 request/token 进入 GPU，让完成的 request 及时退出、新 request 及时补位，提高 GPU utilization 和 tokens/s。

L.3 Chunked Prefill

大 prompt 的 prefill 如果一次吃完，会长时间占据 token budget，阻塞 decode request，造成 TPOT 尖峰。Chunked prefill 把大 prefill 拆成多个 chunk，与 decode token 混合调度，从而在 throughput、TTFT 和 TPOT 间做更细的权衡。

L.4 Paged KV Cache

Paged KV 把逻辑连续 token 的 KV 映射到固定大小 block/page，避免为每个 sequence 预留最大连续空间。好处是降低 external fragmentation，并支持按 block 管理 prefix cache、eviction 和共享。

```
Request A logical blocks: [0][1][2][3]
                        | | | |
Physical KV blocks:      [9][2][7][4] # 不要求连续

Scheduler/Block manager 维护 logical -> physical mapping
```

L.5 Hybrid KV Cache Manager

当前 vLLM 对混合 attention 模型采用分层 KV 管理：KVCacheManager 作为 scheduler 接口；KVCacheCoordinator 协调一个或多个 cache group；SingleTypeKVCacheManager 负责具体 attention type 的 allocation/prefix-cache 逻辑。理解这一层很适合 Staff 面试，因为它体现了“统一调度接口 + 多策略内核”的工程设计。

L.6 Prefix Caching

当大量请求共享 system prompt、长上下文前缀或相同文档前缀时，可以复用已计算的 KV block，减少 prefill compute。命中率高时显著降低 TTFT/compute，但会占用 KV capacity，因此 cache admission/eviction 策略很重要。

L.7 Preemption / KV Pressure

当 KV 不足以继续调度所有 running request，系统可能需要暂停/重计算/驱逐部分 request。这里应关注“可服务并发”的真实约束不是 GPU core，而是 KV memory budget。高 KV utilization 并非永远更好，因为没有 headroom 时长请求突发会造成 preemption storm。

参数/机制	主要影响	典型权衡
max_num_batched_tokens	每 iteration token budget	吞吐 vs latency
max_num_seqs	并发 sequence 上限	吞吐 vs KV/调度开销
gpu_memory_utilization	可给 KV/engine 的显存比例	capacity vs OOM headroom
chunked prefill	prefill 分块	TTFT/TPOT fairness vs 调度复杂度
prefix caching	复用前缀 KV	compute savings vs cache occupancy

L.Q1 : 为什么 vLLM 比“一个请求一个 batch”的 serving 吞吐高？

2 分钟回答

关键不是某一个 kernel，而是调度和内存管理：continuous batching 让 GPU 每个 iteration 都尽量有工作，paged KV cache 降低碎片和过度预留，prefix cache/optimized kernels 再进一步减少重复计算。

- 深入回答：我会从 token scheduling、KV admission 和 GPU execution 三层解释，而不是只说 “用了 PagedAttention”。
- 指标：waiting/running requests、KV usage、prefix hit、TTFT、TPOT、tokens/s
 - Trade-off：追求最大吞吐通常需要更高 batching/KV occupancy，但会增加尾延迟和突发 OOM/preemption 风险。
 - Interviewer 追问：continuous batching 如何影响 TTFT？；paged KV 为什么降低 fragmentation？；KV 满了怎么办？；chunked prefill 解决什么尾延迟问题？

M. LLM Parallelism 与 GPU Topology 决策

M.1 TP / PP / DP / EP

并行	切什么	核心通信	最适场景	主要风险
TP	单层 tensor/matrix	AllReduce/AllGather/ReduceScatter	单模型放不下或需要单请求多卡	高频低延迟通信
PP	模型 layers/stages	点对点 activation	跨节点/模型很大	pipeline bubble、调度复杂
DP	模型 replica	请求路由/训练时梯度同步	提升总吞吐	每 replica 需要完整权重
EP	MoE experts	AllToAll	MoE 模型	fabric bisection BW、load imbalance

M.2 并行策略选择原则

1. 先确定单卡/单节点是否能容纳模型和目标 KV cache。
2. 能单节点则优先把高通信频率的 TP 放在 NVLink/NVSwitch 域内。
3. 跨节点扩展优先考虑 DP/PP，除非必须做跨节点 TP。
4. MoE EP 必须把 AllToAll 网络当成一级设计约束。
5. 用真实 prompt/output length 分布 benchmark，而不是只测固定 128/128。

M.3 Prefill/Decode Disaggregation

Prefill 与 decode 资源特征不同：prefill 大矩阵计算密集，decode 通常更 memory-bandwidth/latency sensitive。Disaggregated prefill 把两阶段放在不同 worker/pool，并通过 KV transfer 连接，可以独立扩缩和优化 TTFT/ITL。但增加了 KV 传输、路由、故障恢复和容量协调复杂度。

N. GPU Observability 与 SLO

N.1 四层指标模型

层	指标	要回答的问题
业务/Inference	QPS、tokens/s、TTFT、TPOT/ITL、E2E latency	用户体验和吞吐是否达标？
Serving Engine	queue depth、running/waiting、batch tokens、KV usage/hit/preemption	调度/KV 是否成为瓶颈？
GPU	SM/Tensor util、HBM used/BW、power/temp、PCIe/NVLink、Xid/ECC	硬件在做什么、是否健康？
Network/NCCL	collective time、busbw/algbw、RDMA throughput、drops/ECN/PFC	通信是否拖慢 GPU？
K8s/Node	pending、allocation、node pressure、device health、restarts	容量与编排是否异常？

N.2 Golden Signals for LLM

- Latency: E2E、TTFT、TPOT/ITL，必须关注 P50/P95/P99。
- Traffic: requests/s、input/output tokens/s，比单纯 QPS 更能描述负载。
- Errors: HTTP errors、OOM、preemption、NCCL failures、GPU Xid。
- Saturation: queue depth、KV cache、GPU memory、SM/HBM BW、NIC/RDMA link。

N.3 为什么只看 GPU Utilization 会误判

GPU utilization 是“是否有 kernel active”的粗粒度指标。它无法区分低效 kernel、memory stall、频繁小 kernel、通信等待，也不能反映用户体验。因此告警和 autoscaling 不应只依赖 GPU util。

N.4 SLO 示例

```
Availability >= 99.9%
P95 TTFT < 800 ms (按 prompt bucket 定义)
P95 TPOT < 50 ms/token
Error rate < 0.1%
Queue wait P95 < 200 ms
GPU unhealthy/Xid node quarantine < 2 min
```

O. 生产故障案例与标准 Runbook

O.1 GPU OOM

症状：流量突增后部分 replica OOM，GPU memory 98%，KV cache >95%。

标准处理：先按 weights/KV/workspace/reserved 拆账；确认请求 context/batch 分布是否改变；限制超长请求并降低 max concurrency；必要时 scale out；长期通过 admission、KV watermark 和 capacity model 防止再次发生。

- 必须保存：时间线、版本、拓扑、关键 metrics、日志、复现条件和最终根因。
- 复盘要求：说明为什么已有监控没有更早发现，以及应该新增什么 guardrail。

O.2 NCCL Hang

症状：TP 多卡请求卡住，GPU util 部分 0%，应用无明显 error。

标准处理：检查所有 rank 是否进入相同 collective；运行 NCCL diagnostics；核对 Xid/ECC/NVLink；nccl-tests 验证节点内；跨节点再检查 RDMA/GID/MTU/PFC/ECN。

- 必须保存：时间线、版本、拓扑、关键 metrics、日志、复现条件和最终根因。
- 复盘要求：说明为什么已有监控没有更早发现，以及应该新增什么 guardrail。

O.3 GPU Xid

症状：某节点间歇性请求失败，dmesg 出现 NVRM Xid。

标准处理：将 GPU/Node quarantine，保留日志与 DCGM diagnostics；根据 Xid 类型判断 driver/GPU/PCIe；不要仅重启 Pod 后继续承载生产。

- 必须保存：时间线、版本、拓扑、关键 metrics、日志、复现条件和最终根因。
- 复盘要求：说明为什么已有监控没有更早发现，以及应该新增什么 guardrail。

O.4 RoCE 吞吐下降

症状：多节点性能突然下降，单卡正常。

标准处理：对比 nccl-tests 与 ib_write_bw；查看 ethtool/交换机 ECN/PFC/drops；检查 MTU、QoS mapping、NIC firmware、route；用拓扑验证是否 scheduler 把 GPU 分到远 NIC。

- 必须保存：时间线、版本、拓扑、关键 metrics、日志、复现条件和最终根因。
- 复盘要求：说明为什么已有监控没有更早发现，以及应该新增什么 guardrail。

O.5 GPU Pending

症状：Pod Pending: insufficient nvidia.com/gpu，但监控显示有空闲 GPU。

标准处理：区分“物理空闲”与“调度已分配”；查看 Node allocatable/Pod requests、MIG profile、taint/affinity/topology；确认 device plugin health 和 Node label。

- 必须保存：时间线、版本、拓扑、关键 metrics、日志、复现条件和最终根因。
- 复盘要求：说明为什么已有监控没有更早发现，以及应该新增什么 guardrail。

O.6 性能回归

症状：新镜像上线后 tokens/s -25%，GPU util 仍高。

标准处理：对比软件版本/driver/CUDA/NCCL/kernel；做 workload replay；Nsight 比较 kernel 与 NCCL 时间；检查是否 quant/kernel backend 变化或 graph capture 失效。

- 必须保存：时间线、版本、拓扑、关键 metrics、日志、复现条件和最终根因。
- 复盘要求：说明为什么已有监控没有更早发现，以及应该新增什么 guardrail。

P. Capacity Engineering：从 tokens 到 GPU 数量

P.1 不要用 QPS 单独做容量

两个 10 QPS workload 可能完全不同：一个 prompt 50/output 20，另一个 prompt 20k/output 2k。LLM capacity 必须把 input tokens、output tokens、context length、并发和 latency target 纳入。

```
 $\lambda_{req}$  = request rate
 $T_{in}$  = avg input tokens/request
 $T_{out}$  = avg output tokens/request
TokenDemand  $\approx \lambda_{req} \times (T_{in} + T_{out})$ 
但 prefill/decode 的每 token 成本不同，因此最好分别建模。
```

P.2 Little's Law

```
L =  $\lambda \times W$ 
平均系统并发 = 到达率  $\times$  平均停留时间
例如 20 req/s  $\times$  5s = 平均约 100 个 in-flight requests
```

Little’ s Law 能把吞吐、延迟、并发联系起来，是推理容量规划非常实用的工具。但 workload 长尾明显时必须用分位数/bucket 模型，而不是只看平均。

P.3 Headroom

- GPU memory headroom：防止长上下文、graph/workspace、碎片化造成 OOM。
- throughput headroom：为突发、降频、单节点故障留余量。
- network headroom：collective 通信不应长期跑到 fabric 极限。
- failure headroom：按 N+1/N+AZ failure 设计，而不是健康状态 100% 填满。

P.4 Autoscaling 信号

信号	优点	缺点	适用
GPU Util	简单	与用户排队不一一对应	辅助指标
Queue Depth/Wait	直接反映供需	需防突发噪声	在线 serving 首选之一
TTFT/TPOT	贴近 SLO	反馈滞后	SLO guardrail
KV Usage	捕捉 memory saturation	不能代表 compute saturation	长上下文 workload
Tokens/s demand	最符合 LLM 工作量	需要精确 instrumentation	成熟平台

Q. Staff-Level GPU Platform System Design

Q.1 设计 1000-GPU 多租推理平台

面试回答应先定义目标：模型数量、模型大小、GPU SKU、region、租户数量、流量分布、TTFT/TPOT SLO、成本目标、数据安全等级，然后再画架构。

```
Clients
-> Global LB / API Gateway
-> Auth / Rate Limit / Admission
-> Model Router
-> Regional Inference Control Plane
    -> Model Registry / Rollout Controller
    -> Queue / Quota / Priority
    -> Autoscaler
    -> Placement / GPU Scheduler
-> Serving Pools (vLLM/TensorRT-LLM)
    -> GPU Nodes / NVLink domains
    -> RDMA fabric
-> Observability + Cost + Audit
```

设计维度	必须讨论
调度	SKU、TP topology、gang、quota、preemption、fragmentation
模型发布	canary、shadow、warmup、rollback、model cache
扩缩容	GPU node cold start、warm pool、queue-aware、predictive
可靠性	GPU quarantine、node failure、AZ capacity、NCCL failure
多租户	namespace/identity/quota、MIG/whole GPU、network/data isolation
成本	tokens/\$、GPU hour utilization、reserved/spot、packing
可观测	request trace -> engine -> GPU -> NCCL/RDMA -> node

Q.2 Model Cold Start

大模型冷启动可能由节点启动、driver 初始化、镜像拉取、模型权重下载、反序列化、GPU H2D copy、NCCL communicator 初始化、CUDA Graph warmup 等组成。优化时必须量化每一段，而不是笼统说“做缓存”。

- Node warm pool; image pre-pull; local NVMe model cache; parallel weight load; GPU Direct Storage（若适用）；提前 NCCL/warmup；模型 replica 预测扩容。

Q.3 Cost Model

```
Cost_per_1M_tokens ≈ (GPU_hour_price × GPU_count × wall_time_hours) / (tokens / 1e6)
同时考虑: idle capacity、node overhead、network、storage、failed/retried requests。
```

R. 最终版面试题库：80 个高频深度问题

R.1 [GPU Architecture] SM 与 CUDA Core/Tensor Core 的关系是什么？

回答框架：从 SM 是执行资源集合开始，解释 warp scheduler、register/shared memory、CUDA/Tensor execution unit。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.2 [GPU Architecture] 为什么 GPU 不适合所有低延迟任务？

回答框架：强调 launch/transfer overhead、SIMT、并行度、单线程 latency 与 CPU 的差异。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.3 [CUDA] Driver API 与 Runtime API 区别？

回答框架：Runtime 更高层，底层仍通过 driver；生产容器中 kernel driver 在 host。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.4 [CUDA] 什么是 CUDA Stream？

回答框架：ordering domain + 并发；不同 stream 是否并发取决于依赖与资源。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.5 [CUDA] CUDA Graph 解决什么问题？

回答框架：减少重复 kernel launch 的 CPU/driver overhead，适合稳定执行图。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.6 [Memory] HBM bandwidth 为什么对 decode 重要？

回答框架：decode batch 较小时每 token 读取大量权重，算术强度低，容易 memory bound。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.7 [Memory] GQA/MQA 为什么省 KV？

回答框架：减少 KV heads 数，从而 KV cache 近似线性下降。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.8 [Memory] allocated 和 reserved 的区别？

回答框架：caching allocator 实际 tensor vs 预留池。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.9 [Memory] Paged KV Cache 怎么降低碎片？

回答框架：逻辑 block 映射到非连续物理 block，避免连续大块预留。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.10 [Interconnect] PCIe、NVLink、NVSwitch 分别解决什么？

回答框架：CPU/device IO、GPU point-to-point high BW、GPU fabric all-to-all connectivity。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.11 [NCCL] AllReduce 为什么常拆成 ReduceScatter + AllGather？

回答框架：解释分片聚合与再收集及 ring 带宽利用。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.12 [NCCL] NCCL Hang 第一件事做什么？

回答框架：确认 rank 一致与 diagnostics，再分 GPU/P2P/RDMA 层排。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.13 [NCCL] algbw 与 busbw 有什么意义？

回答框架：应用观察 bandwidth 与归一化到总线的数据移动效率。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.14 [RDMA] 为什么 GPUDirect RDMA 能更快？

回答框架：绕过 CPU staging/host copy，让 NIC 直接访问 GPU memory。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.15 [RDMA] RoCE 为什么需要 ECN/PFC？

回答框架：以太网拥塞/丢包下 RDMA 性能敏感，需要流控与拥塞反馈。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.16 [K8s] Device Plugin 如何工作？

回答框架：Register/ListAndWatch/Allocate；Node allocatable 到 kubelet device allocation。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.17 [K8s] scheduler 是否知道具体 GPU ID ?

回答框架：传统 extended resource 不一定；通常 kubelet 具体分配。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.18 [K8s] DRA 解决什么问题？

回答框架：设备声明、属性/selector、claim、driver 协作，增强复杂设备表达。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.19 [K8s] 为什么需要 gang scheduling ?

回答框架：避免部分 worker 占住 GPU 等剩余资源造成死锁/浪费。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.20 [K8s] GPU node autoscaling 为什么难？

回答框架：node 冷启动慢、SKU 稀缺、拓扑、模型加载慢、multiple loops。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.21 [Sharing] MIG 与 time slicing 最大区别？

回答框架：硬件资源 partition 与时间共享，隔离/可预测性不同。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.22 [Sharing] 什么时候不该共享 GPU？

回答框架：严格 latency/QoS、安全隔离、高通信 workload。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.23 [Inference] Prefill 和 Decode 的资源特征？

回答框架：prefill 更 compute-friendly；decode 更 latency/memory-bandwidth sensitive。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.24 [Inference] Continuous Batching 为什么有效？

回答框架：iteration 级动态补位，提高 GPU 占用和吞吐。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.25 [Inference] Chunked Prefill 的 trade-off ?

回答框架：降低大 prompt 对 decode 的阻塞，但增加调度和可能延长单请求 prefill。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.26 [Inference] Prefix Cache 什么时候收益最大？

回答框架：大量共享长前缀，减少重复 prefill；受 cache occupancy 限制。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.27 [Inference] KV Cache 满了怎么办？

回答框架：admission/preemption/eviction/recompute/scale out/limit context。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.28 [Parallelism] 为什么优先节点内 TP？

回答框架：NVLink/NVSwitch 低延迟高带宽，避免跨节点 collective。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.29 [Parallelism] MoE 为什么难在网络？

回答框架：EP 引入 AllToAll，且 expert imbalance 导致热点。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.30 [Observability] GPU util 低如何排？

回答框架：queue/CPU/tokenizer/data load/kernel gaps/NCCL/small batch。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.31 [Observability] GPU util 高但吞吐低如何排？

回答框架：memory bound、低效 kernel、通信、功耗/降频、batching。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.32 [Reliability] Xid error 怎么处理？

回答框架：先 quarantine，再收集 DCGM/dmesg/driver/topology；按类型决定 reset/RMA。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.33 [Reliability] 为什么不能只重启 Pod ？

回答框架：硬件/driver/fabric 故障不会被 Pod 重启修复，可能反复污染流量。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.34 [Capacity] 如何估算 GPU 数量 ？

回答框架：按真实 token mix benchmark + SLO + headroom，而不是参数量/QPS 单变量。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.35 [Capacity] 为什么平均 tokens/request 不够 ？

回答框架：长尾 context/out tokens 对 KV 和 service time 非线性影响大。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.36 [System Design] 如何做 model rollout ？

回答框架：权重预热、canary/shadow、指标比较、版本路由、快速 rollback。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.37 [System Design] 如何减少 cold start ？

回答框架：warm nodes/cache/weights/preload/NCCCL/CUDA graph warmup。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.38 [System Design] GPU 平台如何做多租户公平 ？

回答框架：quota/queue/priority/DRF-like fairness + isolation + chargeback。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.39 [System Design] 如何处理 GPU SKU 异构 ？

回答框架：能力标签、DeviceClass、模型硬件兼容矩阵、benchmark-aware routing。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.40 [System Design] 如何评价平台效率 ？

回答框架：SLO 前提下 tokens/\$、GPU-hour utilization、queue wait、failure waste。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.41 [CUDA] 场景题 1：CUDA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.42 [Memory] 场景题 2：Memory 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.43 [NCCL] 场景题 3：NCCL 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.44 [RDMA] 场景题 4：RDMA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.45 [K8s/DRA] 场景题 5：K8s/DRA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.46 [Inference] 场景题 6：Inference 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.47 [Observability] 场景题 7：Observability 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.48 [System Design] 场景题 8：System Design 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.49 [CUDA] 场景题 9：CUDA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.50 [Memory] 场景题 10：Memory 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.51 [NCCL] 场景题 11：NCCL 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.52 [RDMA] 场景题 12：RDMA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.53 [K8s/DRA] 场景题 13：K8s/DRA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.54 [Inference] 场景题 14：Inference 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.55 [Observability] 场景题 15：Observability 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.56 [System Design] 场景题 16：System Design 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.57 [CUDA] 场景题 17：CUDA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.58 [Memory] 场景题 18：Memory 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.59 [NCCL] 场景题 19：NCCL 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.60 [RDMA] 场景题 20：RDMA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.61 [K8s/DRA] 场景题 21：K8s/DRA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.62 [Inference] 场景题 22：Inference 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.63 [Observability] 场景题 23：Observability 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.64 [System Design] 场景题 24：System Design 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.65 [CUDA] 场景题 25：CUDA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.66 [Memory] 场景题 26：Memory 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.67 [NCCL] 场景题 27：NCCL 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.68 [RDMA] 场景题 28：RDMA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.69 [K8s/DRA] 场景题 29：K8s/DRA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.70 [Inference] 场景题 30：Inference 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.71 [Observability] 场景题 31：Observability 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.72 [System Design] 场景题 32：System Design 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.73 [CUDA] 场景题 33：CUDA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.74 [Memory] 场景题 34：Memory 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.75 [NCCL] 场景题 35：NCCL 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.76 [RDMA] 场景题 36：RDMA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.77 [K8s/DRA] 场景题 37：K8s/DRA 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.78 [Inference] 场景题 38：Inference 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.79 [Observability] 场景题 39：Observability 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

R.80 [System Design] 场景题 40：System Design 层出现性能或可靠性异常，你如何系统定位？

回答框架：回答顺序固定为：定义症状和 SLO -> 缩小故障域 -> 获取基线/对照 -> 用指标和实验验证 -> 临时止损 -> 根因 -> 长期 guardrail。

- 追问要求：至少给出 2 个指标、2 个命令/工具、1 个生产 trade-off。

S. 30 个 Hands-on Labs (最终版)

S.1 CUDA Kernel

目标: vector add/matmul, 改变 block size, 记录 kernel time/occupancy。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.2 Streams

目标: copy + compute 多 stream overlap。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.3 CUDA Graph

目标: 比较 graph 前后 launch overhead。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.4 Nsight Systems

目标: 分析一个 vLLM 请求 timeline。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.5 Nsight Compute

目标: 分析 GEMM 或自定义 kernel stall。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.6 Memory

目标: 不同 batch/context 记录 allocated/reserved。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.7 KV Capacity

目标: 用公式预测 KV, 再与 vLLM metrics 对照。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.8 Fragmentation

目标: 制造长短请求混合观察内存碎片。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.9 Topology

目标: `nvidia-smi topo -m + lspci + numactl` 画图。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.10 P2P

目标: `p2pBandwidthLatencyTest` 比较 GPU pair。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.11 NCCL Ring

目标: `nccl-tests all_reduce_perf message size sweep`。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.12 NCCL Diagnostics

目标: 启用 2.31 diagnostics 并记录结果。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.13 NVLink

目标: 比较 NVLink vs PCIe pair。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.14 RDMA

目标: `ib_write_bw/ib_read_bw` 基线。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.15 RoCE Counters

目标: 采集 ECN/PFC/drop/retrans counters。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.16 GPU-NIC Affinity

目标: 不同 GPU/NIC pairing benchmark。

- 产物: 实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答: 瓶颈在哪一层? 证据是什么? 如果生产化, 需要增加什么 guardrail?

S.17 Device Plugin

目标：追踪 Register/ListAndWatch/Allocate log。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.18 GPU Operator

目标：查看 ClusterPolicy 与所有 DaemonSet dependency。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.19 MIG

目标：创建 profile 并调度 workload。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.20 Time Slicing

目标：多 Pod 共享压测公平性。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.21 DRA

目标：DeviceClass/ResourceClaim/ResourceSlice 生命周期。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.22 Gang

目标：用 Kueue/Volcano 做 all-or-nothing workload admission。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.23 GPU Autoscale

目标：queue-aware GPU node scaling。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.24 vLLM Batch

目标：max_num_batched_tokens/max_num_seqs sweep。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.25 Chunked Prefill

目标：长 prompt + decode 混合比较 TTFT/TPOT。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.26 Prefix Cache

目标：相同 system prompt 比较 hit/TTFT。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.27 TP Scale

目标：TP1/2/4/8 scale efficiency。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.28 OOM Runbook

目标：人为触发 KV/OOM 并完成故障报告。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.29 Xid Drill

目标：基于历史/模拟日志做 quarantine automation。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

S.30 System Design

目标：输出 1000 GPU 平台 ADR + architecture + SLO + cost model。

- 产物：实验假设、环境/版本、拓扑、命令、原始数据、图表、结论。
- 必须回答：瓶颈在哪一层？证据是什么？如果生产化，需要增加什么 guardrail？

T. 最终 Cheat Sheet 与 2026 更新

T.1 高频命令

场景	命令/工具
GPU inventory	nvidia-smi -L; nvidia-smi -q
GPU realtime	nvidia-smi dmon
Topology	nvidia-smi topo -m; lspci -tv; numactl --hardware
NVLink	nvidia-smi nvlink -s
DCGM	dcgmi discovery -l; dcgmi diag -r 3
CUDA profile	nsys profile; ncu --set full
NCCL	NCCL_DEBUG=INFO; nccl-tests; NCCL_RUN_DIAGNOSTICS=1
RDMA	rdma link; ibstat; ibv_devinfo; ib_write_bw
NIC	ethtool -S; ip -s link
K8s GPU	kubectl describe node; kubectl get pods -A -o wide
DRA	kubectl get deviceclasses,resourceclaims,resourceslices
Node logs	journalctl -u kubelet; dmesg -T

T.2 2026 需要特注的方向

- Kubernetes DRA 已进入 stable 阶段，GPU/NIC/accelerator 的声明式设备管理成为核心趋势。
- NCCL 2.31 引入更系统的 RAS/Active Diagnostics，集群级通信故障排查正在从经验式转向结构化诊断。
- GPU Operator 25.10 系列继续强化 Driver/Toolkit/Device Plugin/DCGM/MIG 等全栈生命周期管理，并推进 CDI 等标准化容器设备注入。
- vLLM 的 KV cache 管理正在适配 hybrid attention 模型，Scheduler/KV manager 已成为推理平台面试非常值得深入的源码模块。

T.3 最终学习路径

Week 1: GPU architecture + HBM + CUDA
Week 2: CUDA profiling + memory allocator + topology
Week 3: NCCL + NVLink + RDMA/RoCE
Week 4: Kubernetes Device Plugin + GPU Operator + DRA
Week 5: vLLM + KV Cache + continuous batching
Week 6: TP/PP/DP/EP + capacity/autoscaling
Week 7: observability + failure drills
Week 8: system design + mock interview

最终要求
真正达到 Senior/Staff 水平的判断标准：给你一个“慢、OOM、Hang、Pending、成本高”的模糊症状，你能够跨 CUDA/GPU/NCCL/RDMA/Kubernetes/vLLM 建立假设、选择指标和实验、定位故障域，并说明长期设计 trade-off。